

UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO
CENTRO TECNOLÓGICO
CURSO DE ENGENHARIA DE COMPUTAÇÃO

DÉBORA AZEVEDO CAETANO
GABRIELA BUTERI BELMOCK
GESSICA DE SOUZA SILVA
MARCOS VINICIUS DE SOUZA SILVA

MÃO ROBÓTICA COM MOVIMENTAÇÃO DE PULSO

Relatório técnico do projeto

Vitória - ES

2026

Débora Azevedo Caetano
Gabriela Buteri Belmock
Gessica de Souza Silva
Marcos Vinicius de Souza Silva

MÃO ROBÓTICA COM MOVIMENTAÇÃO DE PULSO

Relatório técnico apresentado à disciplina de Projeto Integrado em Computação II (PIC2) do curso de Engenharia de Computação da Universidade Federal do Espírito Santo, como requisito parcial de avaliação.

Orientadores: Rodolfo da Silva Villaça e Ricardo Carminati de Mello.

Vitória - ES

2026

SUMÁRIO

1 INTRODUÇÃO	5
1.1 Justificativa	5
1.2 Objetivos	6
1.3 Repositório do projeto.....	6
2 VISÃO GERAL DO SISTEMA.....	7
2.1 Arquitetura	7
2.2 Fluxo de funcionamento	8
3 MATERIAIS E EQUIPAMENTOS.....	9
3.1 Microcontrolador.....	9
3.2 Atuadores	9
3.3 Alimentação e componentes eletrônicos	9
3.4 Estrutura mecânica e materiais	10
4 PROJETO DE HARDWARE.....	11
4.1 Mapa de ligações (pinos)	11
4.2 Comunicação Bluetooth (BLE)	11
4.3 Montagem eletrônica e projeto de placas.....	12
5 MODELOS 3D E FABRICAÇÃO	13
5.1 Peças da mão e dos dedos	13
5.2 Peças do pulso e do antebraço	13
5.3 Suporte	14
5.4 Impressão e montagem	14
6 SOFTWARE E FIRMWARE.....	15
6.1 Aplicativo de interface (Flutter)	15
6.2 Servidor intermediário (Python).....	16
6.3 Inteligência: interpretação por LLM	16
6.4 Firmware embarcado (C++)	17
6.5 Biblioteca de configurações de mão	17
7 DIFICULDADES DE DESENVOLVIMENTO	18
7.1 Fio de nylon	18
7.2 Molas de tração	18
7.3 Roldanas	19
7.4 Impressão e estrutura.....	19

8 CONSIDERAÇÕES FINAIS.....	20
REFERÊNCIAS	21
APÊNDICE A - CÓDIGO-FONTE DO APLICATIVO (FLUTTER/DART)	22
APÊNDICE B - CÓDIGO-FONTE DO SERVIDOR (PYTHON).....	37
APÊNDICE C - FIRMWARE EMBARCADO (C++)	44
APÊNDICE D - FIRMWARE DE TESTES POR SERIAL (C++)	48
APÊNDICE E - PROTÓTIPO INICIAL DO SERVIDOR (FLASK).....	55

1 INTRODUÇÃO

A comunicação por meio da Língua Brasileira de Sinais (LIBRAS) é um direito das pessoas surdas e um pilar de acessibilidade. Entretanto, dispositivos assistivos capazes de reproduzir sinais manuais ainda são caros e pouco acessíveis. Este relatório descreve o desenvolvimento de uma **mão robótica de baixo custo**, com cinco dedos funcionais e movimentação de pulso, capaz de reproduzir configurações de mão (CMs) empregadas em LIBRAS a partir de comandos em linguagem natural, fornecidos por voz ou por texto.

O sistema integra hardware e software: uma estrutura mecânica impressa em 3D, acionada por seis servomotores; um microcontrolador que executa os movimentos; um servidor que interpreta a intenção do usuário com o apoio de um modelo de linguagem (LLM) executado localmente; e um aplicativo móvel que serve de interface. O protótipo encontra-se **finalizado e funcional**.

Figura 1 – Protótipo final montado



Fonte: os autores (2026).

1.1 Justificativa

Soluções comerciais de mãos robóticas e próteses ativas possuem custo elevado, o que limita seu uso em contextos educacionais e sociais. A proposta deste projeto é demonstrar que, com **hardware aberto, impressão 3D e software livre**, é possível construir um dispositivo

funcional voltado à comunicação e à acessibilidade a uma fração do custo das alternativas de mercado, servindo também como plataforma didática de robótica e sistemas embarcados.

1.2 Objetivos

O objetivo geral é projetar e construir uma mão robótica capaz de reproduzir configurações de mão da LIBRAS a partir de comandos em linguagem natural. Como objetivos específicos, destacam-se: (i) desenvolver a estrutura mecânica impressa em 3D com dedos e pulso articulados; (ii) implementar o firmware de controle dos servomotores; (iii) desenvolver o servidor de interpretação e a integração com um modelo de linguagem local; (iv) construir o aplicativo de interface com entrada por voz e por texto; e (v) integrar todas as camadas em um fluxo funcional ponta a ponta.

1.3 Repositório do projeto

Todo o código-fonte, os modelos 3D e a documentação estão disponíveis publicamente no repositório:

github.com/gk27min/Mao-Robotica-com-Movimentacao-de-Pulso

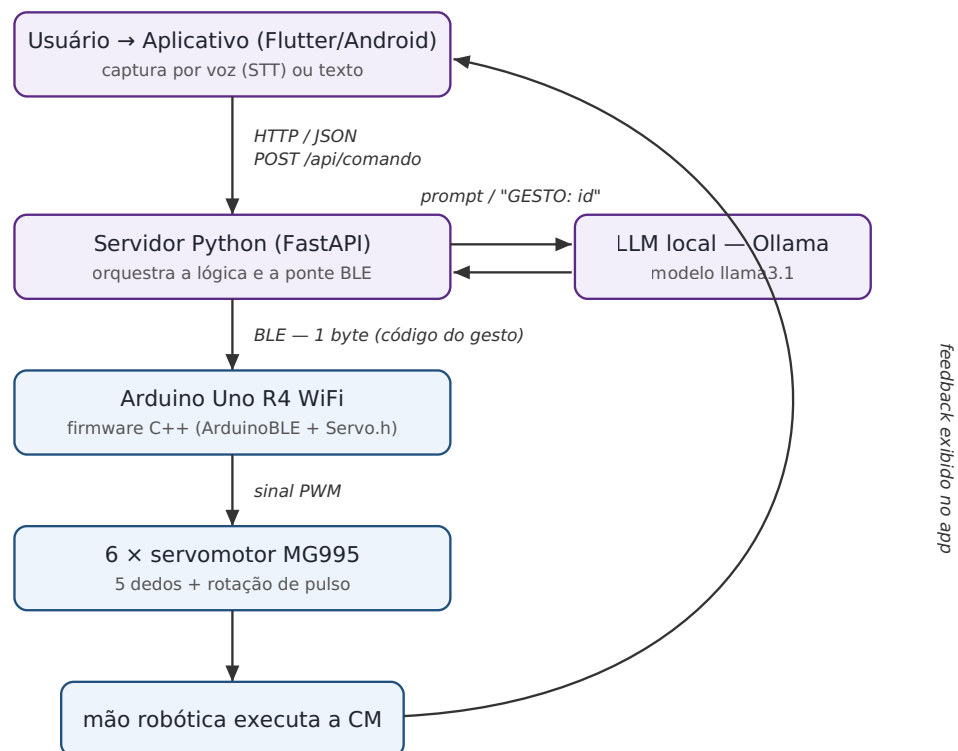
2 VISÃO GERAL DO SISTEMA

O sistema é organizado em camadas, da interface com o usuário até o acionamento físico dos motores. Cada camada tem uma responsabilidade bem definida e se comunica com a seguinte por um protocolo específico, o que facilita o teste e a manutenção de cada parte de forma isolada.

2.1 Arquitetura

A arquitetura adotada é apresentada na Figura 2. O aplicativo captura a fala ou o texto do usuário e envia a frase, por HTTP, ao servidor Python. O servidor consulta um modelo de linguagem (LLM) executado localmente, que classifica a intenção e devolve o identificador do gesto correspondente. Em seguida, o servidor transmite esse identificador, por Bluetooth Low Energy (BLE), ao microcontrolador, que aciona os servomotores por modulação por largura de pulso (PWM).

Figura 2 – Arquitetura do sistema



Fonte: os autores (2026).

2.2 Fluxo de funcionamento

O funcionamento pode ser resumido em cinco etapas. Na *captura*, o aplicativo obtém a fala (por reconhecimento de voz) ou o texto digitado. Na *interpretação*, o servidor envia a frase ao LLM, que escolhe um único gesto da biblioteca e responde no formato padronizado GESTO: <id>; caso o modelo não responda, um mecanismo de reserva por palavras-chave assume a decisão. Na *transmissão*, o servidor envia o código do gesto (um único byte) ao Arduino por BLE. Na *atuação*, o firmware decodifica o byte, calcula os ângulos de cada dedo e do pulso e comanda os motores; ao final, a mão retorna à posição de descanso. Por fim, no *retorno*, o aplicativo exibe o andamento da operação ao usuário.

3 MATERIAIS E EQUIPAMENTOS

Esta seção relaciona os equipamentos, dispositivos eletrônicos, atuadores e materiais empregados na construção do protótipo.

3.1 Microcontrolador

O controle de baixo nível é realizado por um **Arduino Uno R4 WiFi**. Essa placa foi escolhida por oferecer **Bluetooth Low Energy (BLE) nativo**, o que dispensa módulos externos de comunicação sem fio e permite que o servidor envie os comandos diretamente à placa. O microcontrolador é responsável por receber o código do gesto, calcular os ângulos e gerar os sinais PWM que acionam os servomotores.

3.2 Atuadores

Os movimentos são produzidos por **seis servomotores MG995**, de engrenagens metálicas: cinco deles tracionam os dedos (polegar, indicador, médio, anelar e mínimo) e um é responsável pela rotação/ oscilação do pulso. Os servos MG995 foram adotados por conciliarem torque adequado à tração dos dedos e baixo custo. Cada servo opera na faixa de 0° a 180°, controlado por PWM a partir da biblioteca Servo.h.

3.3 Alimentação e componentes eletrônicos

Como seis servos sob carga demandam picos de corrente superiores ao que a porta USB fornece, os motores são alimentados por uma **fonte externa dedicada**, mantendo-se o *GND* comum com o Arduino. As ligações de sinal e de alimentação são distribuídas em uma *proto board*, por meio de cabos *jumper*. O Quadro 1 relaciona os componentes eletrônicos.

Quadro 1 – Equipamentos e dispositivos eletrônicos

Item	Especificação	Qtd.	Função
Microcontrolador	Arduino Uno R4 WiFi	1	Controle e comunicação BLE
Servomotores	MG995 (engrenagem metálica)	6	5 dedos + pulso (PWM)
Fonte de alimentação	Fonte chaveada (5-6 V, corrente p/ 6 servos)	1	Alimentação dos servos
Protoboard	Padrão	1	Distribuição de sinais e energia
Cabos jumper	Macho-macho / macho-fêmea	-	Ligações elétricas
Cabo USB	USB	1	Programação do Arduino
Smartphone Android	Aparelho físico	1	Executa o aplicativo (captura de voz)
Computador	PC/notebook com BLE	1	Executa o servidor e o LLM

Fonte: os autores (2026).

3.4 Estrutura mecânica e materiais

A estrutura é impressa em filamento **PETG** e utiliza fio de nylon para a tração dos dedos, molas de tração para o retorno e roldanas acopladas aos servos. O Quadro 2 relaciona os materiais mecânicos. As especificações detalhadas das molas — cuja definição foi uma das principais dificuldades do projeto — são apresentadas na Seção 7.2.

Quadro 2 – Materiais da estrutura mecânica

Item	Especificação	Qtd.
Filamento	PETG (preto)	conforme impressão
Fio de nylon	Nylon grosso (tração $\approx$ 50 kg)	1 rolo
Molas de tração — dedos	Aço carbono (ver Seção 7.2)	4
Mola de tração — polegar	Aço carbono (ver Seção 7.2)	1
Roldanas / polias	1 impressa sob medida + roldanas dos servos	-
Parafusos	Diversos (fixação)	-

Fonte: os autores (2026).

4 PROJETO DE HARDWARE

4.1 Mapa de ligações (pinos)

Cada servomotor é conectado a um pino digital do Arduino, conforme o Quadro 3. Em razão da montagem mecânica, a lógica de acionamento é *invertida* para parte dos dedos: mínimo, anelar e médio ficam fechados a 180° e abertos a 0°, enquanto o indicador adota o comportamento oposto (fechado a 0° e aberto a 180°). Essa convenção está implementada na função `configurarDedos()` do firmware (Apêndice C).

Quadro 3 – Mapa de pinos (servomotores → Arduino)

Servomotor	Pino digital	Função
Pulso	7	Rotação/oscilação do pulso
Mínimo	8	Dedo mínimo
Anelar	9	Dedo anelar
Médio	10	Dedo médio
Indicador	11	Dedo indicador
Polegar	12	Polegar

Fonte: os autores (2026).

4.2 Comunicação Bluetooth (BLE)

A comunicação entre o servidor e o Arduino é feita por Bluetooth Low Energy, seguindo o modelo GATT. O firmware publica um serviço e uma característica, na qual o servidor escreve um único byte correspondente ao código do gesto. O Quadro 4 apresenta os parâmetros de comunicação.

Quadro 4 – Parâmetros da comunicação BLE

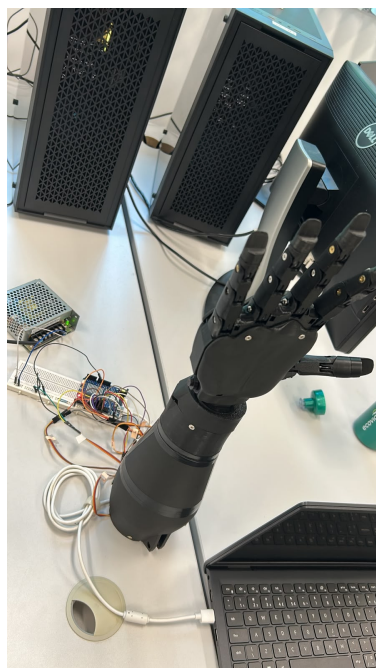
Nome do dispositivo	MaoRobotica
UUID do serviço	19b10000-e8f2-537e-4f6c-d104768a1214
UUID da característica (comando)	19b10001-e8f2-537e-4f6c-d104768a1214
Payload	1 byte (código do gesto)

Fonte: os autores (2026).

4.3 Montagem eletrônica e projeto de placas

O protótipo não emprega uma placa de circuito impresso (PCI) dedicada: a montagem eletrônica foi realizada em *protoboard*, o que favorece a prototipagem, os testes e eventuais ajustes de ligação. A Figura 3 mostra a montagem, com o Arduino, a protoboard, a fonte de alimentação e o computador que executa o servidor. A migração para uma PCI dedicada é apontada como trabalho futuro para aumentar a robustez das conexões.

Figura 3 - Montagem eletrônica na bancada



Fonte: os autores (2026).

5 MODELOS 3D E FABRICAÇÃO

A estrutura mecânica é totalmente impressa em 3D. Os modelos partem do projeto de hardware aberto **InMoov** e receberam adaptações próprias da equipe, descritas na Seção 7. Todos os arquivos .stl utilizados estão versionados no repositório, em docs/3d_parts/. A Figura 4 mostra o conjunto de peças impressas antes da montagem.

Figura 4 – Peças impressas em 3D (PETG)



Fonte: os autores (2026).

5.1 Peças da mão e dos dedos

Compreendem as falanges, as pontas dos dedos, os moldes, as tampas e a estrutura da palma. Arquivos: i2_FingersX5V2, i2_FingersTipX5V2, i2_FingersMoldX5V3, i2_CoverFingerV3, i2_HandCoverV1, i2_HandCoverV2, i2_PalmCoverV2, i2_WristGearV1 e i2_WristLargeV2.

5.2 Peças do pulso e do antebraço

Compreendem o mecanismo de rotação do pulso, o berço do servo, as engrenagens e as guias de cabo. Arquivos: robpart2V4, robpart3V4, robpart4V4, robpart5V4, robcap3V2, RobRingV3, RobServoBedV6, RobCableFrontV3, RobCableBackV3, RotaWrist1V4, RotaWrist2V3, RotaWrist3V3, WristGearsV5 e Bolt_entretoise7.

5.3 Suporte

Base de apoio para exposição e testes do conjunto: Hand_stand.3mf.

5.4 Impressão e montagem

As peças foram impressas em PETG, material escolhido por sua resistência mecânica e tenacidade superiores às do PLA. Após a impressão, parte dos encaixes precisou ser lixada para o ajuste correto. A montagem envolve a passagem do fio de nylon pelos dedos, a instalação das molas de retorno e o acoplamento das roldanas aos servos — etapas detalhadas na Seção 7.

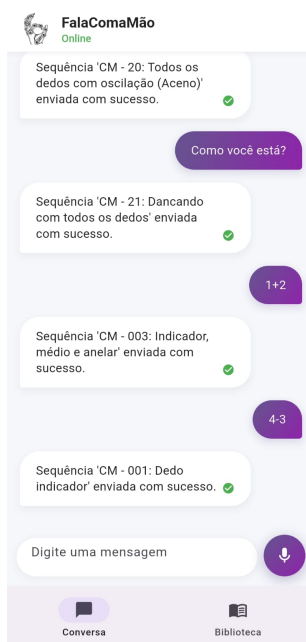
6 SOFTWARE E FIRMWARE

O software está dividido em três blocos: o aplicativo de interface, o servidor intermediário e o firmware embarcado. Os códigos completos, associados ao respectivo hardware de destino, encontram-se nos Apêndices A a E.

6.1 Aplicativo de interface (Flutter)

O aplicativo é desenvolvido em **Flutter (Dart)** e executa em smartphone Android. Possui duas telas: a de *conversa*, no formato de *chat*, com entrada por voz (reconhecimento por meio do pacote `speech_to_text`) e por texto; e a de *biblioteca*, que exibe todas as configurações de mão em uma grade, com ícones animados. A comunicação com o servidor é feita por HTTP, usando o pacote `http`. As Figuras 5 e 6 apresentam as duas telas. O código completo está no Apêndice A.

Figura 5 – Aplicativo: tela de conversa



Fonte: os autores (2026).

Figura 6 – Aplicativo: biblioteca de configurações de mão



Fonte: os autores (2026).

6.2 Servidor intermediário (Python)

O servidor é escrito em **Python**, com o *framework* FastAPI, e executa no computador. Ele expõe dois *endpoints*: `/api/comando`, que recebe uma frase em linguagem natural e a submete ao LLM; e `/api/gesto`, que recebe diretamente um código de gesto (utilizado pela tela de biblioteca). O servidor mantém uma conexão BLE persistente com o Arduino, com reconexão automática em caso de queda. O código completo está no Apêndice B.

6.3 Inteligência: interpretação por LLM

A interpretação da linguagem natural é feita por um modelo de linguagem executado **localmente** por meio do Ollama (modelo llama3.1), sem dependência de serviços em nuvem. Por engenharia de *prompt*, o modelo é instruído a escolher exatamente um gesto da biblioteca e a responder em formato padronizado, sendo ainda capaz de resolver contas simples (por exemplo, "1 + 1" resulta na configuração de dois dedos). Quando o modelo não retorna um gesto válido, um mecanismo de reserva baseado em palavras-chave assume a decisão, aumentando a robustez do sistema.

6.4 Firmware embarcado (C++)

O firmware, escrito em **C++** para o Arduino Uno R4 WiFi, utiliza as bibliotecas `ArduinoBLE` e `Servo.h`. Ao receber um byte pela característica BLE, ele decodifica o código (valores acima de 100 acionam também a oscilação do pulso), resolve os ângulos de cada servo e executa o movimento, retornando a mão à posição de descanso ao final. O código completo está no Apêndice C; um segundo firmware, de testes por porta serial, consta no Apêndice D.

6.5 Biblioteca de configurações de mão

Os códigos das configurações de mão (CMs) são idênticos nas três camadas (aplicativo, servidor e firmware), garantindo coerência de ponta a ponta. Os códigos de 0 a 19 correspondem a gestos estáticos; os iguais ou superiores a 100 acionam adicionalmente a oscilação do pulso. O Quadro 5 apresenta um resumo.

Quadro 5 – Exemplos de configurações de mão (CMs)

Código	Configuração	Exemplos de comando
0	Mão fechada (punho)	"fecha a mão"; "3 – 3"
2	Indicador e médio	"paz"; "1 + 1"
5	Todos os dedos (mão aberta)	"abre a mão"; "cinco"
6	Polegar, indicador e mínimo	"te amo"
7	Polegar	"joinha"; "positivo"
13	Médio, anelar e mínimo	"ok"; "beleza"
18	Sinal de água (indicador batendo)	"água"; "beber"
20 / 105	Mão aberta com oscilação (aceno)	"oi"; "tchau"

Fonte: os autores (2026).

7 DIFICULDADES DE DESENVOLVIMENTO

Esta seção registra os principais obstáculos enfrentados e as soluções adotadas. Trata-se de informação essencial para quem pretenda reproduzir o projeto.

7.1 Fio de nylon

A qualidade do movimento dos dedos mostrou-se sensível ao tipo de fio de tração. Fios finos não conferiam firmeza suficiente. A solução foi adotar um **fio de nylon mais grosso**, com resistência de tração da ordem de **50 kg**, com o qual a movimentação dos dedos ficou adequada.

7.2 Molas de tração

As molas de retorno possuem dimensões muito específicas e **não foram encontradas no mercado**, sendo necessário mandá-las fabricar. A definição das propriedades ideais foi um desafio: ganchos com angulação diferente de 0° ou 180° deixavam os dedos desalinhados e o movimento irregular. As especificações que funcionaram são apresentadas nos Quadros 6 e 7.

Quadro 6 – Especificação das molas dos dedos (quantidade: 4)

Diâmetro do arame (fio)	0,5 mm
Diâmetro externo	4,8 mm
Altura do corpo (sem ganchos)	46 mm
Altura total (com ganchos)	56 mm
Posição dos ganchos	alinhados (180°)
Material	aço carbono
Galvanizada	não

Fonte: os autores (2026).

Quadro 7 – Especificação da mola do polegar (quantidade: 1)

Diâmetro do arame (fio)	0,5 mm
Diâmetro externo	4,8 mm
Altura do corpo (sem ganchos)	42,5 mm
Altura total (com ganchos)	52 mm
Posição dos ganchos	alinhados (180°)
Material	aço carbono
Galvanizada	não

Fonte: os autores (2026).

7.3 Roldanas

A impressão da roldana não resultou em um encaixe perfeito no eixo do servo. A solução foi **fabricar uma roldana que se acopla em torno da roldana original** que acompanha o motor, obtendo-se então um ajuste firme e preciso.

7.4 Impressão e estrutura

Diversos encaixes precisaram ser lixados após a impressão. As partes centrais do braço não possuem parafusos, o que consideramos um ponto negativo por reduzir a rigidez estrutural. Além disso, a caixa do servo do pulso não é bem estruturada e possui uma saliência que chegou a prender o fio de nylon de um dos dedos, tensionando-o em excesso e provocando a **queima de um dos motores**. Recomenda-se revisar e reforçar essa peça antes de operar o conjunto sob carga.

8 CONSIDERAÇÕES FINAIS

O projeto atingiu seus objetivos: foi construído um protótipo funcional de mão robótica capaz de reproduzir configurações de mão da LIBRAS a partir de comandos de voz ou texto, integrando estrutura mecânica impressa em 3D, firmware embarcado, servidor de interpretação com LLM local e aplicativo móvel. As dificuldades enfrentadas — sobretudo relativas ao fio de nylon, às molas sob medida e às roldanas — foram superadas e estão documentadas para permitir a reprodução do trabalho. Como trabalhos futuros, sugere-se a migração da eletrônica para uma placa de circuito impresso dedicada, o reforço da caixa do pulso e a ampliação da biblioteca de gestos.

REFERÊNCIAS

ARDUINO. *Arduino UNO R4 WiFi documentation*. Disponível em: <https://docs.arduino.cc>. Acesso em: 2026.

FLUTTER. *Flutter documentation*. Disponível em: <https://docs.flutter.dev>. Acesso em: 2026.

INMOOV. *InMoov - Open-source 3D printed life-size robot*. Disponível em: <https://inmoov.fr>. Acesso em: 2026.

OLLAMA. *Ollama*. Disponível em: <https://ollama.com>. Acesso em: 2026.

APÊNDICE A - CÓDIGO-FONTE DO APLICATIVO (FLUTTER/DART)

Hardware de destino: smartphone Android. Arquivo: app/lib/main.dart.

```
import 'dart:convert';
import 'dart:math' as math;
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:speech_to_text/speech_to_text.dart' as stt;
import 'package:http/http.dart' as http;

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  SystemChrome.setEnabledSystemUIMode(SystemUiMode.immersiveSticky);
  runApp(const FalaComaMaoApp());
}

Future<bool> enviarGestoDiretoParaServidor(int comando) async {
  const String serverUrl = 'http://192.168.1.4:5000/api/gesto'; // <-- /api/gesto
  final url = Uri.parse(serverUrl);

  try {
    final response = await http.post(
      url,
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({'comando': comando}),
    ).timeout(const Duration(seconds: 15));

    if (response.statusCode == 200) {
      print("Gesto $comando enviado com sucesso!");
      return true;
    } else {
      print("Erro no servidor (${response.statusCode}): ${response.body}");
      return false;
    }
  } catch (e) {
    print("Erro ao enviar gesto: $e");
    return false;
  }
}

class FalaComaMaoApp extends StatelessWidget {
  const FalaComaMaoApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'FalaComaMão',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6200EA), // Cor vibrante
          brightness: Brightness.light,
        ),
        useMaterial3: true,
      ),
      home: const MainNavigationScreen(),
      debugShowCheckedModeBanner: false,
    );
  }
}

// =====
// NAVEGAÇÃO PRINCIPAL (Conversa / Biblioteca)
// =====
class MainNavigationScreen extends StatefulWidget {
  const MainNavigationScreen({super.key});

  @override
  State<MainNavigationScreen> createState() => _MainNavigationScreenState();
}

class _MainNavigationScreenState extends State<MainNavigationScreen> {
  int _selectedIndex = 0;

  static const List<Widget> _screens = [
```

```

    ChatScreen(),
    SignsLibraryScreen(),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: IndexedStack(index: _selectedIndex, children: _screens),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _selectedIndex,
        onDestinationSelected: (index) => setState(() => _selectedIndex = index),
        destinations: const [
          NavigationDestination(
            icon: Icon(Icons.chat_bubble_outline_rounded),
            selectedIcon: Icon(Icons.chat_bubble_rounded),
            label: 'Conversa',
          ),
          NavigationDestination(
            icon: Icon(Icons.menu_book_outlined),
            selectedIcon: Icon(Icons.menu_book_rounded),
            label: 'Biblioteca',
          ),
        ],
      ),
    );
  }
}

// Classe estrutural para as mensagens
class ChatMessage {
  String text;
  final bool isUser;
  String status; // 'none', 'analyzing', 'done', 'error'

  ChatMessage({
    required this.text,
    required this.isUser,
    this.status = 'none',
  });
}

class ChatScreen extends StatefulWidget {
  const ChatScreen({super.key});

  @override
  State<ChatScreen> createState() => _ChatScreenState();
}

class _ChatScreenState extends State<ChatScreen> {
  late stt.SpeechToText _speech;
  bool _isListening = false;
  bool _isProcessing = false;
  String _recognizedText = '';

  final TextEditingController _textController = TextEditingController();
  final ScrollController _scrollController = ScrollController();

  final List<ChatMessage> _messages = [
    ChatMessage(text: 'Oi, Tudo Bem', isUser: false),
  ];

  @override
  void initState() {
    super.initState();
    _speech = stt.SpeechToText();
    _textController.addListener(() => setState(() {}));
  }

  @override
  void dispose() {
    _textController.dispose();
    _scrollController.dispose();
    super.dispose();
  }

  void _scrollToBottom() {
    WidgetsBinding.instance.addPostFrameCallback((_) {

```

```

        if (!_scrollController.hasClients) return;
        _scrollController.animateTo(
          _scrollController.position.maxScrollExtent,
          duration: const Duration(milliseconds: 250),
          curve: Curves.easeOut,
        );
      });
    }
  }

void _processFinalText(String text) {
  if (text.isEmpty) return;

  // 1. Adiciona a mensagem do usuário e a resposta provisória do robô
  setState(() {
    _messages.add(ChatMessage(text: text, isUser: true));
    _messages.add(ChatMessage(
      text: 'Analisando...',
      isUser: false,
      status: 'analyzing',
    ));
    _recognizedText = ''; // Limpa o buffer de reconhecimento
  });
  _scrollToBottom();

  // 2. Chama a API passando o índice exato da mensagem do robô que deve ser atualizada
  final robotMessageIndex = _messages.length - 1;
  enviarComandoParaServidor(text, robotMessageIndex);
}

void _sendTextMessage() {
  if (_isProcessing) return;
  final text = _textController.text.trim();
  if (text.isEmpty) return;
  _textController.clear();
  _isProcessing = true;
  _processFinalText(text);
}

void _cancelListening() {
  HapticFeedback.lightImpact();
  _speech.cancel();
  setState(() {
    _isListening = false;
    _recognizedText = '';
  });
}

void _listen() async {
  // Bloqueia interações se já estiver processando
  if (_isProcessing) return;

  if (!_isListening) {
    bool available = await _speech.initialize(
      onStatus: (val) {
        if (val == 'done' || val == 'notListening') {
          setState(() => _isListening = false);
          // Removido disparo manual. A responsabilidade agora é apenas do finalResult.
        }
      },
      onError: (val) {
        print('onError: $val');
        setState(() => _isListening = false);
      },
    );

    if (available) {
      HapticFeedback.lightImpact();
      setState(() {
        _isListening = true;
        _recognizedText = '';
      });
      _speech.listen(
        onResult: (val) {
          setState(() {
            _recognizedText = val.recognizedWords;

```



```

    });
    // Trava de segurança: somente dispara se for finalResult E não estiver processando
    if (val.finalResult && !_isProcessing) {
        setState(() => _isListening = false);
        _isProcessing = true; // Trava o envio duplo
        _processFinalText(val.recognizedWords);
    }
    },
    );
}
} else {
    HapticFeedback.mediumImpact();
    setState(() => _isListening = false);
    _speech.stop();
    // Removido disparo manual. A responsabilidade agora é apenas do finalResult.
}
}

// Endereço IP do servidor Python na rede local.
static const String _serverUrl = 'http://192.168.1.4:5000/api/comando';

Future<void> enviarComandoParaServidor(String texto, int messageIndex) async {
    final url = Uri.parse(_serverUrl);

    try {
        final response = await http.post(
            url,
            headers: {'Content-Type': 'application/json'},
            body: jsonEncode({'texto': texto}),
        ).timeout(const Duration(seconds: 60));

        if (!mounted) return;

        if (response.statusCode == 200) {
            final data = jsonDecode(response.body) as Map<String, dynamic>;
            final sucesso = data['sucesso'] as bool;
            final detalhes = data['detalhes'] as String;
            final gestureName = data['comando_detectado'] as String?;

            setState(() {
                _isProcessing = false;
                if (sucesso && gestureName != null) {
                    // Mostra primeiro que a mão está atuando; o texto final substitui em seguida.
                    _messages[messageIndex].text = 'Mão executando o gesto: $
{gestureName.toUpperCase()}...';
                    _messages[messageIndex].status = 'acting';
                } else {
                    _messages[messageIndex].text = sucesso ? detalhes : 'Não reconheci. $detalhes';
                    _messages[messageIndex].status = sucesso ? 'done' : 'error';
                }
            });
            _scrollToBottom();

            if (sucesso && gestureName != null) {
                _finishHandActing(messageIndex, detalhes);
            } else if (!sucesso) {
                HapticFeedback.vibrate();
            }
        } else {
            setState(() {
                _messages[messageIndex].text = 'Erro no servidor (${response.statusCode}).';
                _messages[messageIndex].status = 'error';
                _isProcessing = false;
            });
            HapticFeedback.vibrate();
        }
    } catch (e) {
        if (!mounted) return;
        setState(() {
            _messages[messageIndex].text = 'Servidor inacessível. Verifique a conexão.';
            _messages[messageIndex].status = 'error';
            _isProcessing = false;
        });
        HapticFeedback.vibrate();
    }
}
}

```

```

void _finishHandActing(int messageIndex, String detalhes) {
  HapticFeedback.mediumImpact();
  // O servidor confirma apenas o envio do comando via Bluetooth, não o término do
  // movimento físico – sem telemetria do Arduino, a duração aqui é uma estimativa de UX.
  Future.delayed(const Duration(seconds: 3), () {
    if (!mounted) return;
    if (_messages[messageIndex].status != 'acting') return;
    setState(() {
      _messages[messageIndex].text = detalhes;
      _messages[messageIndex].status = 'done';
    });
    _scrollToBottom();
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFF5F7FB),
    appBar: AppBar(
      backgroundColor: Colors.white,
      elevation: 0,
      surfaceTintColor: Colors.transparent,
      shadowColor: Colors.black12,
      title: Row(
        children: [
          const CircleAvatar(
            backgroundImage: AssetImage('assets/icon.jpg'),
            backgroundColor: Colors.transparent,
          ),
          const SizedBox(width: 12),
          const Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                'FalaComaMão',
                style: TextStyle(
                  color: Colors.black87,
                  fontSize: 18,
                  fontWeight: FontWeight.bold,
                ),
              ),
              Text(
                'Online',
                style: TextStyle(
                  color: Colors.green,
                  fontSize: 13,
                  fontWeight: FontWeight.w500,
                ),
              ),
            ],
          ),
        ],
      ),
    ),
    bottom: PreferredSize(
      preferredSize: const Size.fromHeight(1.0),
      child: Container(
        color: Colors.grey[200],
        height: 1.0,
      ),
    ),
    body: Column(
      children: [
        Expanded(
          child: ListView.builder(
            controller: _scrollController,
            padding: const EdgeInsets.all(16),
            itemCount: _messages.length + ((_isListening || _recognizedText.isNotEmpty) ? 1 :
0),
            itemBuilder: (context, index) {
              // Mensagens renderizadas do histórico
              if (index < _messages.length) {
                final msg = _messages[index];
                return ChatBubble(
                  message: msg.text,
                  isMe: msg.isUser,

```

```

        status: msg.status,
      );
    } else {
      // A última bolha é flutuante (tempo real) para o que o usuário está falando
      if (_isListening && _recognizedText.isEmpty) {
        return const ChatBubble(
          child: TypingIndicator(),
          isMe: true,
        );
      } else {
        return ChatBubble(
          message: _recognizedText,
          isMe: true,
        );
      }
    }
  },
),
),
SafeArea(
  top: false,
  child: Padding(
    padding: const EdgeInsets.fromLTRB(12, 4, 12, 12),
    child: AnimatedSwitcher(
      duration: const Duration(milliseconds: 200),
      child: _isListening ? _buildRecordingBar() : _buildComposerBar(),
    ),
  ),
),
),
),
),
),
);
}

```

```

Widget _buildComposerBar() {
  final hasText = _textController.text.trim().isNotEmpty;
  return Row(
    key: const ValueKey('composer'),
    crossAxisAlignment: CrossAxisAlignment.end,
    children: [
      Expanded(
        child: Container(
          constraints: const BoxConstraints(minHeight: 48, maxHeight: 120),
          padding: const EdgeInsets.symmetric(horizontal: 18, vertical: 6),
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius: BorderRadius.circular(28),
            boxShadow: [
              BoxShadow(
                color: Colors.black.withOpacity(0.05),
                blurRadius: 8,
                offset: const Offset(0, 2),
              ),
            ],
          ),
        child: TextField(
          controller: _textController,
          minLines: 1,
          maxLines: 4,
          enabled: !_isProcessing,
          textCapitalization: TextCapitalization.sentences,
          decoration: const InputDecoration(
            hintText: 'Digite uma mensagem',
            border: InputBorder.none,
            isCollapsed: true,
          ),
          onSubmitted: (_) => _sendTextMessage(),
        ),
      ),
      const SizedBox(width: 8),
      GestureDetector(
        onTap: _isProcessing ? null : (hasText ? _sendTextMessage : _listen),
        child: AnimatedContainer(
          duration: const Duration(milliseconds: 200),
          width: 52,
          height: 52,

```

```

        decoration: BoxDecoration(
          shape: BoxShape.circle,
          gradient: LinearGradient(
            colors: _isProcessing
              ? [Colors.grey.shade400, Colors.grey.shade500]
              : [Theme.of(context).colorScheme.primary, const Color(0xFF8E24AA)],
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
          ),
          boxShadow: [
            BoxShadow(
              color: Theme.of(context).colorScheme.primary.withOpacity(0.35),
              blurRadius: 12,
              offset: const Offset(0, 4),
            ),
          ],
        ),
        child: Icon(
          hasText ? Icons.send_rounded : Icons.mic_rounded,
          color: Colors.white,
        ),
      ),
    ),
  ],
);
}

Widget _buildRecordingBar() {
  return Container(
    key: const ValueKey('recording'),
    padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 6),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(28),
      boxShadow: [
        BoxShadow(
          color: Colors.black.withOpacity(0.05),
          blurRadius: 8,
          offset: const Offset(0, 2),
        ),
      ],
    ),
    child: Row(
      children: [
        IconButton(
          icon: const Icon(Icons.delete_outline_rounded, color: Colors.black45),
          onPressed: _cancelListening,
          tooltip: 'Cancelar',
        ),
        Expanded(
          child: _recognizedText.isEmpty
            ? const Padding(
                padding: EdgeInsets.symmetric(horizontal: 8),
                child: WaveformIndicator(),
              )
            : Padding(
                padding: const EdgeInsets.symmetric(horizontal: 8),
                child: Text(
                  _recognizedText,
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                  style: const TextStyle(color: Colors.black87),
                ),
              ),
        ),
        const Padding(
          padding: EdgeInsets.symmetric(horizontal: 4),
          child: Text('Ouvindo...', style: TextStyle(color: Colors.black45, fontSize: 12)),
        ),
        GestureDetector(
          onTap: _listen,
          child: Container(
            width: 44,
            height: 44,
            decoration: const BoxDecoration(
              shape: BoxShape.circle,
              gradient: LinearGradient(colors: [Colors.redAccent, Colors.deepOrange]),
            ),

```

```

        ),
        child: const Icon(Icons.stop_rounded, color: Colors.white),
      ),
    ),
  ],
),
);
}
}

class ChatBubble extends StatelessWidget {
  final String? message;
  final Widget? child;
  final bool isMe;
  final String status;

  const ChatBubble({
    super.key,
    this.message,
    this.child,
    required this.isMe,
    this.status = 'none',
  });

  @override
  Widget build(BuildContext context) {
    final textStyle = TextStyle(
      color: isMe ? Colors.white : Colors.black87,
      fontSize: 15,
      height: 1.3,
    );

    // Constrói o conteúdo do balão com base no status da resposta do robô
    Widget content;
    if (child != null) {
      content = child!;
    } else {
      if (status == 'analyzing') {
        content = Row(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            const SizedBox(
              width: 16,
              height: 16,
              child: CircularProgressIndicator(
                strokeWidth: 2,
                color: Color(0xFF8E24AA), // Cor vibrante igual ao tema principal
              ),
            ),
            const SizedBox(width: 12),
            Flexible(child: Text(message ?? '', style: textStyle)),
          ],
        );
      } else if (status == 'acting') {
        content = Row(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            const Icon(Icons.back_hand, size: 18, color: Color(0xFF8E24AA)),
            const SizedBox(width: 10),
            Flexible(child: Text(message ?? '', style: textStyle)),
          ],
        );
      } else if (status == 'done') {
        content = Row(
          mainAxisAlignment: MainAxisAlignment.min,
          crossAxisAlignment: CrossAxisAlignment.end,
          children: [
            Flexible(child: Text(message ?? '', style: textStyle)),
            const SizedBox(width: 6),
            const Icon(Icons.check_circle_rounded, size: 16, color: Colors.green),
          ],
        );
      } else if (status == 'error') {
        content = Row(
          mainAxisAlignment: MainAxisAlignment.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [

```

```

        const Icon(Icons.error_outline_rounded, size: 18, color: Colors.redAccent),
        const SizedBox(width: 8),
        Flexible(child: Text(message ?? '', style: textStyle)),
      ],
    );
  } else {
    content = Text(message ?? '', style: textStyle);
  }
}

return Align(
  alignment: isMe ? Alignment.centerRight : Alignment.centerLeft,
  child: Container(
    margin: const EdgeInsets.only(bottom: 16),
    padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 14),
    constraints: BoxConstraints(
      maxWidth: MediaQuery.of(context).size.width * 0.75,
      minHeight: 48,
    ),
    decoration: BoxDecoration(
      gradient: isMe
        ? LinearGradient(
            colors: [
              Theme.of(context).colorScheme.primary,
              const Color(0xFF8E24AA),
            ],
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
          )
        : null,
      color: isMe ? null : (status == 'error' ? const Color(0xFFFFF3F3) : Colors.white),
      borderRadius: BorderRadius.only(
        topLeft: const Radius.circular(24),
        topRight: const Radius.circular(24),
        bottomLeft: Radius.circular(isMe ? 24 : 4),
        bottomRight: Radius.circular(isMe ? 4 : 24),
      ),
      border: status == 'error' ? Border.all(color: Colors.redAccent.withOpacity(0.3)) : null,
      boxShadow: [
        BoxShadow(
          color: isMe
            ? Theme.of(context).colorScheme.primary.withOpacity(0.2)
            : Colors.black.withOpacity(0.05),
          blurRadius: 8,
          offset: const Offset(0, 4),
        ),
      ],
    ),
    child: content,
  ),
);
}
}

class TypingIndicator extends StatefulWidget {
  const TypingIndicator({super.key});

  @override
  State<TypingIndicator> createState() => _TypingIndicatorState();
}

class _TypingIndicatorState extends State<TypingIndicator> with SingleTickerProviderStateMixin {
  late AnimationController _controller;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 1200),
    )..repeat();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }
}

```

```

}

@override
Widget build(BuildContext context) {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 4),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.min,
      children: List.generate(3, (index) {
        return AnimatedBuilder(
          animation: _controller,
          builder: (context, child) {
            final delay = index * 0.2;
            var val = (_controller.value - delay) % 1.0;
            if (val < 0) val += 1.0;

            final offset = (val < 0.5)
              ? Curves.easeInOut.transform(val * 2)
              : Curves.easeInOut.transform((1 - val) * 2);

            return Transform.translate(
              offset: Offset(0, -6 * offset),
              child: child,
            );
          },
        ),
      ),
    ),
  );
}

// Indicador de forma de onda animado, usado na barra de gravação de áudio.
class WaveformIndicator extends StatefulWidget {
  const WaveformIndicator({super.key});

  @override
  State<WaveformIndicator> createState() => _WaveformIndicatorState();
}

class _WaveformIndicatorState extends State<WaveformIndicator> with SingleTickerProviderStateMixin {
  late final AnimationController _controller;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 900),
    )..repeat();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      height: 24,
      child: Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: List.generate(5, (i) {
          return AnimatedBuilder(
            animation: _controller,

```

```

        builder: (context, child) {
          final t = (_controller.value + i * 0.18) % 1.0;
          final height = 6 + 14 * (0.5 + 0.5 * math.sin(t * 2 * math.pi));
          return Container(
            margin: const EdgeInsets.symmetric(horizontal: 2),
            width: 4,
            height: height,
            decoration: BoxDecoration(
              color: const Color(0xFF8E24AA),
              borderRadius: BorderRadius.circular(2),
            ),
          );
        },
      ),
    ),
  ),
);
}

class SignsLibraryScreen extends StatelessWidget {
  const SignsLibraryScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // Lista 100% atualizada e espelhada com o backend e o Arduino (0 a 19 e 100 a 119)
    final List<Map<String, dynamic>> configuracoesMao = [
      // --- BLOCO 1: ESTÁTICOS (Pulso Fixo) ---
      {'comando': 0, 'titulo': 'CM - 000', 'descricao': 'Mão totalmente fechada'},
      {'comando': 1, 'titulo': 'CM - 001', 'descricao': 'Dedo indicador'},
      {'comando': 2, 'titulo': 'CM - 002', 'descricao': 'Indicador e médio'},
      {'comando': 3, 'titulo': 'CM - 003', 'descricao': 'Indicador, médio e anelar'},
      {'comando': 4, 'titulo': 'CM - 004', 'descricao': 'Indicador a mindinho'},
      {'comando': 5, 'titulo': 'CM - 005', 'descricao': 'Todos os dedos'},
      {'comando': 6, 'titulo': 'CM - 006', 'descricao': 'Polegar, indicador e mindinho'},
      {'comando': 7, 'titulo': 'CM - 007', 'descricao': 'Polegar'},
      {'comando': 8, 'titulo': 'CM - 008', 'descricao': 'Dedo médio'},
      {'comando': 9, 'titulo': 'CM - 009', 'descricao': 'Polegar e mindinho'},
      {'comando': 10, 'titulo': 'CM - 010', 'descricao': 'Indicador e mindinho'},
      {'comando': 11, 'titulo': 'CM - 011', 'descricao': 'Mindinho'},
      {'comando': 12, 'titulo': 'CM - 012', 'descricao': 'Polegar e indicador'},
      {'comando': 13, 'titulo': 'CM - 013', 'descricao': 'Médio, anelar e mindinho'},
      {'comando': 14, 'titulo': 'CM - 014', 'descricao': 'Letra C (Dedos curvados)'},
      {'comando': 15, 'titulo': 'CM - 015', 'descricao': 'Letra A (Polegar lateral)'},
      {'comando': 16, 'titulo': 'CM - 016', 'descricao': 'Letra O (Círculo)'},
      {'comando': 17, 'titulo': 'CM - 017', 'descricao': 'Base para Letra H'},

      // --- BLOCO 1.1: ESTÁTICOS COM MOVIMENTO DE DEDO ---
      {'comando': 18, 'titulo': 'CM - 018', 'descricao': 'Sinal de Água (Indicador batendo)'},
      {'comando': 19, 'titulo': 'CM - 019', 'descricao': 'Sinal de Aspas (Indicador e médio dobrando)'},

      // --- BLOCO 2: DINÂMICOS (Com oscilação do pulso) ---
      {'comando': 100, 'titulo': 'CM - 100', 'descricao': 'Mão fechada'},
      {'comando': 101, 'titulo': 'CM - 101', 'descricao': 'Indicador (Não)'},
      {'comando': 102, 'titulo': 'CM - 102', 'descricao': 'Indicador e médio'},
      {'comando': 103, 'titulo': 'CM - 103', 'descricao': 'Três dedos'},
      {'comando': 104, 'titulo': 'CM - 104', 'descricao': 'Quatro dedos'},
      {'comando': 105, 'titulo': 'CM - 105', 'descricao': 'Mão aberta (Aceno)'},
      {'comando': 106, 'titulo': 'CM - 106', 'descricao': 'Te amo balançando'},
      {'comando': 107, 'titulo': 'CM - 107', 'descricao': 'Polegar'},
      {'comando': 108, 'titulo': 'CM - 108', 'descricao': 'Dedo médio'},
      {'comando': 109, 'titulo': 'CM - 109', 'descricao': 'Shaka balançando'},
      {'comando': 110, 'titulo': 'CM - 110', 'descricao': 'Rock balançando'},
      {'comando': 111, 'titulo': 'CM - 111', 'descricao': 'Mindinho'},
      {'comando': 112, 'titulo': 'CM - 112', 'descricao': 'Polegar e indicador'},
      {'comando': 113, 'titulo': 'CM - 113', 'descricao': 'Médio, anelar, mindinho balançando'},
      {'comando': 114, 'titulo': 'CM - 114', 'descricao': 'Letra C balançando'},
      {'comando': 115, 'titulo': 'CM - 115', 'descricao': 'Letra A balançando'},
      {'comando': 116, 'titulo': 'CM - 116', 'descricao': 'Letra O balançando'},
      {'comando': 117, 'titulo': 'CM - 117', 'descricao': 'Letra H (Base com rotação)'},
      {'comando': 118, 'titulo': 'CM - 118', 'descricao': 'Sinal de Água do pulso'},
      {'comando': 119, 'titulo': 'CM - 119', 'descricao': 'Sinal de Aspas do pulso'},
    ];

    return Scaffold(
      appBar: AppBar(

```



```

        title: const Text('Biblioteca de Gestos'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),
      body: GridView.builder(
        padding: const EdgeInsets.all(16),
        gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent(
          maxCrossAxisExtent: 200,
          crossAxisSpacing: 16,
          mainAxisSpacing: 16,
          childAspectRatio: 0.85,
        ),
        itemCount: configuracoesMao.length,
        itemBuilder: (context, index) {
          final cm = configuracoesMao[index];

          return InkWell(
            onTap: () async {
              final messenger = ScaffoldMessenger.of(context); // captura antes do await
              final titulo = cm['titulo'] as String;

              messenger.showSnackBar(
                SnackBar(content: Text('Enviando $titulo...')),
              );

              final ok = await enviarGestoDiretoParaServidor(cm['comando'] as int);

              messenger.hideCurrentSnackBar();
              messenger.showSnackBar(
                SnackBar(
                  content: Text(ok ? '$titulo enviado!' : 'Falha ao enviar $titulo'),
                  backgroundColor: ok ? Colors.green : Colors.red,
                ),
              );
            },
            child: Card(
              elevation: 2,
              color: Colors.white,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(16),
                side: BorderSide(color: Colors.grey.shade200, width: 1),
              ),
              child: Padding(
                padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 12),
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    HandGestureIcon(
                      gestureLevel: cm['comando'] as int,
                      size: 80,
                    ),
                    const SizedBox(height: 16),
                    Text(
                      cm['titulo'] as String,
                      style: const TextStyle(fontWeight: FontWeight.w800, fontSize: 15),
                    ),
                    const SizedBox(height: 6),
                    Text(
                      cm['descricao'] as String,
                      textAlign: TextAlign.center,
                      style: TextStyle(color: Colors.grey.shade600, fontSize: 11),
                    ),
                  ],
                ),
              ),
            ),
          );
        },
      ),
    ),
  );
}

class RegisterSignScreen extends StatelessWidget {
  const RegisterSignScreen({super.key});

  @override

```

```

Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Cadastrar Sinal'),
      backgroundColor: Theme.of(context).colorScheme.primary,
      foregroundColor: Colors.white,
    ),
    body: const Center(
      child: Text('Formulário de cadastro', style: TextStyle(fontSize: 18)),
    ),
  );
}

// Transformado em StatefulWidget para suportar o Loop de animação dos dedos
class HandGestureIcon extends StatefulWidget {
  final int gestureLevel;
  final double size;

  const HandGestureIcon({
    super.key,
    required this.gestureLevel,
    this.size = 100.0,
  });

  @override
  State<HandGestureIcon> createState() => _HandGestureIconState();
}

class _HandGestureIconState extends State<HandGestureIcon> with SingleTickerProviderStateMixin {
  late AnimationController _fingerController;

  @override
  void initState() {
    super.initState();
    // Controlador de animação que vai e volta (looping infinito)
    _fingerController = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 500),
    )..repeat(reverse: true);
  }

  @override
  void dispose() {
    _fingerController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final activeColor = Theme.of(context).colorScheme.primary;
    final inactiveColor = Colors.grey.shade300;

    final int baseGesture = widget.gestureLevel % 100;
    final bool hasWristMovement = widget.gestureLevel >= 100;

    // Estados dos dedos: 0 = Fechado, 1 = Aberto, 2 = Curvado (metade), 3 = Batendo (loop)
    int ind = 0, mei = 0, ane = 0, min = 0, pol = 0;

    switch (baseGesture) {
      case 1: ind = 1; break;
      case 2: ind = 1; mei = 1; break;
      case 3: ind = 1; mei = 1; ane = 1; break;
      case 4: ind = 1; mei = 1; ane = 1; min = 1; break;
      case 5: ind = 1; mei = 1; ane = 1; min = 1; pol = 1; break;
      case 6: ind = 1; min = 1; pol = 1; break;
      case 7: pol = 1; break;
      case 8: mei = 1; break;
      case 9: min = 1; pol = 1; break;
      case 10: ind = 1; min = 1; break;
      case 11: min = 1; break;
      case 12: ind = 1; pol = 1; break;
      case 13: mei = 1; ane = 1; min = 1; break;
      case 14: ind = 2; mei = 2; ane = 2; min = 2; pol = 2; break; // Letra C (Curvados)
      case 15: pol = 2; break; // Letra A (Apenas polegar lateral relaxado)
      case 16: ind = 0; mei = 2; ane = 2; min = 2; pol = 2; break; // Letra O (Círculo)
      case 17: ind = 1; pol = 2; break; // Base H (Indicador aberto, polegar metade)
    }
  }
}

```

```

        case 18: ind = 3; break; // Água (Indicador batendo repetidamente)
        case 19: ind = 3; mei = 3; break; // Aspas (Indicador e médio batendo)
    }

    final fw = widget.size * 0.13;
    final palmWidth = widget.size * 0.55;
    final palmHeight = widget.size * 0.50;

    return SizedBox(
      width: widget.size,
      height: widget.size,
      child: Stack(
        clipBehavior: Clip.none,
        children: [
          // Polegar
          Positioned(
            bottom: widget.size * 0.18,
            right: widget.size * 0.15,
            child: Transform.rotate(
              angle: 0.78,
              alignment: Alignment.bottomLeft,
              child: _buildRoboticFinger(pol, activeColor, inactiveColor, fw, widget.size * 0.35),
            ),
          ),
          // Chassi da Mão
          Positioned(
            bottom: widget.size * 0.05,
            left: widget.size * 0.20,
            child: Container(
              width: palmWidth,
              height: palmHeight,
              decoration: BoxDecoration(
                color: activeColor,
                borderRadius: BorderRadius.circular(widget.size * 0.1),
              ),
            ),
          ),
          // Os 4 dedos superiores
          Positioned(
            bottom: widget.size * 0.52,
            left: widget.size * 0.20,
            child: SizedBox(
              width: palmWidth,
              child: Row(
                mainAxisAlignment: MainAxisAlignment.spaceEvenly,
                crossAxisAlignment: CrossAxisAlignment.end,
                children: [
                  _buildRoboticFinger(min, activeColor, inactiveColor, fw, widget.size * 0.40),
                  _buildRoboticFinger(ane, activeColor, inactiveColor, fw, widget.size * 0.45),
                  _buildRoboticFinger(me, activeColor, inactiveColor, fw, widget.size * 0.50),
                  _buildRoboticFinger(ind, activeColor, inactiveColor, fw, widget.size * 0.40),
                ],
              ),
            ),
          ),
          // Indicador de Movimento do Pulso
          if (hasWristMovement)
            Positioned(
              top: -widget.size * 0.15,
              right: -widget.size * 0.15,
              child: Icon(
                Icons.autorenew_rounded,
                color: activeColor,
                size: widget.size * 0.30,
              ),
            ),
        ],
      ),
    );
}

// O desenho de cada dedo agora reage aos 4 estados
Widget _buildRoboticFinger(int state, Color activeColor, Color inactiveColor, double width,
double maxHeight) {

```

```

return AnimatedBuilder(
  animation: _fingerController,
  builder: (context, child) {
    double currentHeight = maxHeight * 0.25; // Altura base (Fechado)
    Color currentColor = inactiveColor;

    if (state == 1) {
      // Totalmente Aberto
      currentHeight = maxHeight;
      currentColor = activeColor;
    } else if (state == 2) {
      // Curvado (Metade da altura) - Efeito de pintura pela metade
      currentHeight = maxHeight * 0.55;
      currentColor = activeColor;
    } else if (state == 3) {
      // Batendo (Loop que varia a altura do dedo para cima e para baixo)
      // Varia de 25% (fechado) a 100% (aberto)
      currentHeight = maxHeight * (0.25 + (0.75 * _fingerController.value));
      // Faz a cor acender e apagar durante a batida
      currentColor = Color.lerp(inactiveColor, activeColor, _fingerController.value) ??
activeColor;
    }

    return Container(
      width: width,
      height: currentHeight,
      decoration: BoxDecoration(
        color: currentColor,
        borderRadius: BorderRadius.circular(width / 2),
        border: Border.all(
          color: Colors.white,
          width: 1.5,
        ),
      ),
    );
  }
);
}
}

```

Dependências (arquivo app/pubspec.yaml): flutter, speech_to_text ^7.0.0, http ^1.1.0, cupertino_icons ^1.0.8.

APÊNDICE B - CÓDIGO-FONTE DO SERVIDOR (PYTHON)

Hardware de destino: computador (PC/notebook). Servidor de produção (FastAPI + BLE).

Arquivo: backend/server.py.

```
import logging
import uvicorn
import asyncio
from typing import Optional
from contextlib import asynccontextmanager # 1. ADICIONADO AQUI
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

# Importação dos módulos internos
from brain import MapeadorDeSinais, GESTOS
from bluetooth_sender import enviar_comando_bluetooth, conectar_bluetooth, desconectar_bluetooth

# Configuração de Logs
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("server")

# 2. COLOQUE A FUNÇÃO LIFESPAN AQUI (Substituindo os antigos @app.on_event)
@asynccontextmanager
async def lifespan(app: FastAPI):
    # 0 que acontece no startup:
    await conectar_bluetooth()
    yield
    # 0 que acontece no shutdown:
    await desconectar_bluetooth()

# 3. PASSAR O LIFESPAN DENTRO DO FASTAPI
app = FastAPI(
    title="LIBRAS-BOT Backend",
    description="Servidor intermediário em Python (FalaComaMão) para processamento de voz e ponte BLE.",
    version="1.0.0",
    lifespan=lifespan # ADICIONADO AQUI
)

# Configuração de CORS (Cross-Origin Resource Sharing)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Inicialização do Mapeador de Sinais (Zero-DB)
mapeador = MapeadorDeSinais()

# =====
# MODELOS DE DADOS (Pydantic)
# =====
class ComandoRequest(BaseModel):
    texto: str

class GestoRequest(BaseModel):
    comando: int

class ComandoResponse(BaseModel):
    sucesso: bool
    texto_original: str
    comando_detectado: Optional[str] = None
    codigo: Optional[int] = None
    confianca: float
    detalhes: str

# =====
# ENDPOINTS
# =====
```

```

@app.get("/")
def read_root():
    return {"status": "online", "servico": "LIBRAS-BOT Backend"}

@app.post("/api/comando", response_model=ComandoResponse)
async def processar_comando(requisicao: ComandoRequest):
    texto = requisicao.texto
    logger.info(f"Requisição recebida com texto de voz: '{texto}'")

    # 1. 'codigos' agora recebe uma LISTA
    codigos, descricao, similaridade = await mapeador.processar_comando(texto)

    if not codigos: # Se a lista estiver vazia
        detalhes = "Comando não compreendido pela IA."
        return ComandoResponse(sucesso=False, texto_original=texto, confianca=0.0,
                                detalhes=detalhes)

    logger.info(f"Comandos compreendidos: {descricao} (Códigos: {codigos})")

    # 2. Camada de Comunicação: Envia a sequência com delay
    for cod in codigos:
        if cod == -1:
            logger.warning("Sinal de erro/inválido ignorado na sequência.")
            continue

        envio_sucesso = await enviar_comando_bluetooth(cod)

        if not envio_sucesso:
            # Se falhar no meio da sequência, aborta o resto
            detalhes = f"Falha ao enviar o código {cod} via Bluetooth."
            logger.error(detalhes)
            return ComandoResponse(
                sucesso=False, texto_original=texto, comando_detectado=descricao,
                confianca=similaridade, detalhes=detalhes
            )

        # Pausa vital para o Arduino executar o movimento físico (4 segundos)
        # Usar asyncio.sleep previne que o servidor inteiro trave durante a pausa
        await asyncio.sleep(4)

    # 3. Retorno com Sucesso
    detalhes = f"Sequência '{descricao}' enviada com sucesso."
    return ComandoResponse(
        sucesso=True, texto_original=texto, comando_detectado=descricao, confianca=similaridade,
        detalhes=detalhes
    )

@app.post("/api/gesto", response_model=ComandoResponse)
async def enviar_gesto_direto(requisicao: GestoRequest):
    codigo = requisicao.comando
    logger.info(f"Gesto direto recebido da biblioteca: {codigo}")

    # Valida se o código existe na biblioteca
    if codigo not in GESTOS:
        detalhes = f"Código de gesto {codigo} não existe na biblioteca."
        return ComandoResponse(
            sucesso=False, texto_original=str(codigo), confianca=0.0, detalhes=detalhes
        )

    descricao = GESTOS[codigo][0]

    # Envia direto pro Bluetooth, sem passar pela LLM
    envio_sucesso = await enviar_comando_bluetooth(codigo)

    if not envio_sucesso:
        detalhes = f"Falha ao enviar o gesto {codigo} via Bluetooth."
        return ComandoResponse(
            sucesso=False, texto_original=str(codigo), comando_detectado=descricao,
            codigo=codigo, confianca=1.0, detalhes=detalhes
        )

    detalhes = f"Gesto '{descricao}' enviado com sucesso."
    return ComandoResponse(
        sucesso=True, texto_original=str(codigo), comando_detectado=descricao,
        codigo=codigo, confianca=1.0, detalhes=detalhes
    )

```

```
if __name__ == "__main__":
    uvicorn.run("server:app", host="0.0.0.0", port=5000, reload=True)
```

Arquivo: *backend/brain.py*.

```
import re
import logging
import ollama
from typing import Optional, Tuple

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("brain")

# Gestos disponíveis: ID → (Descrição semântica e física, palavras-chave para fallback)
GESTOS = {
    # =====
    # BLOCO 1: ESTÁTICOS (Pulso Fixo) -> 0 a 17
    # =====
    0: ("CM - 000: Mão totalmente fechada", ["fechar", "punho", "zero", "fechada"]),
    1: ("CM - 001: Dedo indicador", ["apontar", "indicador", "ponto", "direção", "um", "1"]),
    2: ("CM - 002: Indicador e médio", ["paz", "vitoria", "vitória", "peace", "victory"]),
    3: ("CM - 003: Indicador, médio e anelar", ["tres", "três"]),
    4: ("CM - 004: Indicador, médio, anelar e mindinho", ["quatro"]),
    5: ("CM - 005: Todos os dedos", ["abrir", "aberta", "estender", "todos", "dedos", "cinco",
    "espalmar", "pare", "descanso", "repouso", "posição inicial"]),
    6: ("CM - 006: Polegar, indicador e mindinho", ["amo", "love", "te amo", "ily"]),
    7: ("CM - 007: Polegar", ["joinha", "positivo", "bom", "legal", "like", "curtir", "polegar",
    "valeu"]),
    8: ("CM - 008: Dedo médio", ["dedo do meio", "ofensivo", "médio", "xingamento"]),
    9: ("CM - 009: Polegar e mindinho", ["hanglose", "hang loose", "shaka", "surf", "surfista",
    "telefone", "alô"]),
    10: ("CM - 010: Indicador e mindinho", ["rock", "metal", "chifres", "horns"]),
    11: ("CM - 011: Mindinho", ["letra i", "libras i", "mindinho"]),
    12: ("CM - 012: Polegar e indicador", ["letra l", "libras l", "loser", "faz o l"]),
    13: ("CM - 013: Médio, anelar e mindinho", ["ok", "certo", "tudo certo", "concordo", "beleza",
    "perfeito"]),
    14: ("CM - 014: Letra C em LIBRAS (Dedos curvados)", ["letra c", "libras c"]),
    15: ("CM - 015: Letra A em LIBRAS", ["letra a", "libras a"]),
    16: ("CM - 016: Letra O em LIBRAS (Dedos formando um círculo)", ["letra o", "libras o", "zero
    libras"]),
    17: ("CM - 017: Base para Letra H (Indicador e médio estendidos, polegar lateral)", ["base
    h"]),

    # =====
    # BLOCO 1.1: ESTÁTICOS COM MOV. DE DEDOS -> 18 a 20
    # =====
    18: ("CM - 018: Sinal de Água (Indicador batendo repetidas vezes)", ["agua", "água", "sede",
    "beber"]),
    19: ("CM - 019: Sinal de Aspas (Indicador e médio dobrando repetidas vezes)", ["aspas",
    "citação", "quote", "entre aspas"]),
    20: ("CM - 105: Todos os dedos com oscilação (Aceno)", ["aceno", "acenar", "tchau", "oi",
    "olá", "ola", "hello", "wave", "saudação", "cumprimentar", "e aí", "e aí"]),
    # 20: ("CM - 020: Reservado para futuros", ["reservado"]),
    # 22: ("CM - 117: Letra H em LIBRAS (Base do H com rotação do pulso)", ["letra h", "libras
    h"]),

    # =====
    # BLOCO 2: DINÂMICOS (Com oscilação do pulso) -> 100 a 120
    # =====
    100: ("CM - 100: Mão totalmente fechada com oscilação", ["bater", "soco", "bater na
    porta"]),
    101: ("CM - 101: Dedo indicador com oscilação (Sinal de Não)", ["nao", "não", "negativo",
    "recusa", "discordo", "rejeitar"]),
    102: ("CM - 102: Indicador e médio com oscilação", ["observar", "olhar"]),
    103: ("CM - 103: Indicador, médio e anelar com oscilação", ["tres balançando"]),
    104: ("CM - 104: Indicador, médio, anelar e mindinho com oscilação", ["quatro balançando"]),
    106: ("CM - 106: Polegar, indicador e mindinho com oscilação", ["te amo balançando", "ily
    com movimento"]),
    107: ("CM - 107: Polegar com oscilação", ["joinha balançando", "positivo balançando"]),
    108: ("CM - 108: Dedo médio com oscilação", ["médio balançando"]),
    109: ("CM - 109: Polegar e mindinho com oscilação", ["shaka balançando", "telefone
    balançando"]),
    110: ("CM - 110: Indicador e mindinho com oscilação", ["rock balançando"]),
    111: ("CM - 111: Mindinho com oscilação", ["mindinho balançando"]),
    112: ("CM - 112: Polegar e indicador com oscilação", ["letra l balançando"]),
    113: ("CM - 113: Médio, anelar e mindinho com oscilação", ["ok balançando"]),
```

```

    # 114: ("CM - 114: Letra C com oscilação do pulso", ["c balançando"]),
    # 115: ("CM - 115: Letra A com oscilação do pulso", ["a balançando"]),
    # 116: ("CM - 116: Letra O com oscilação do pulso", ["o balançando"]),
    # 118: ("CM - 118: Sinal de Água com oscilação do pulso", ["agua balançando"]),
    # 119: ("CM - 119: Sinal de Aspas com oscilação do pulso", ["aspas balançando"]),
    # 120: ("CM - 120: Reservado com oscilação", ["reservado balançando"]),
}

LLM_MODEL = "llama3.1"

# Bloco de texto que descreve os gestos, reutilizado no prompt do LLM
_GESTOS_PROMPT = "\n".join(
    f" ID {gid}: {desc}" for gid, (desc, _) in GESTOS.items()
)

_PROMPT_TEMPLATE = """\
Você é o controlador de uma mão robótica de LIBRAS.
Sua tarefa: dada a frase do usuário, escolher O ÚNICO gesto da lista abaixo
que melhor representa o que a frase quer dizer.
REGRA EXTRA: Se o usuário pedir o resultado de uma soma ou subtração simples, resolva a conta
matematicamente. Se o resultado for de 0 a 5, retorne o ID correspondente ao resultado numérico.

Sua ÚNICA saída permitida é uma destas duas formas:
- "GESTO: <ID>" (usando um ID que EXISTE na lista abaixo)
- "NENHUM" (quando nada na lista se encaixa na frase, ou se o resultado da conta for menor
que 0 ou maior que 5)
PROIBIDO escrever qualquer outra coisa, explicação ou texto extra.

GESTOS DISPONÍVEIS (ID: Descrição):
{gestos}

EXEMPLOS (siga exatamente este formato):
Entrada: "Oi"
Saída: GESTO: 105

Entrada: "Quanto é 1 + 1?"
Saída: GESTO: 2

Entrada: "3 - 3"
Saída: GESTO: 0

Entrada: "soma 2 mais 3"
Saída: GESTO: 5

Entrada: "Tchau, até mais"
Saída: GESTO: 105

Entrada: "Abre a mão"
Saída: GESTO: 5

Entrada: "Fecha a mão"
Saída: GESTO: 0

Entrada: "Manda um joinha"
Saída: GESTO: 7

Entrada: "Te amo"
Saída: GESTO: 6

Entrada: "Faz o sinal de paz"
Saída: GESTO: 2

Entrada: "Não"
Saída: GESTO: 101

Entrada: "asdfghjkl"
Saída: NENHUM

Entrada: "5 + 5"
Saída: NENHUM

Entrada: "{texto}"
Saída: ""

class MapeadorDeSinais:
    def __init__(self, llm_model: str = LLM_MODEL):

```



```

self.llm_model = llm_model
logger.info(f"MapeadorDeSinais iniciado com modelo '{self.llm_model}'.")

async def _classificar_com_llm(self, texto: str) -> Optional[int]:
    prompt = _PROMPT_TEMPLATE.format(gestos=_GESTOS_PROMPT, texto=texto)
    try:
        client = ollama.AsyncClient()
        response = await client.generate(
            model=self.llm_model,
            prompt=prompt,
            options={
                "temperature": 0.0,
                "top_p": 0.1,
                "num_predict": 10, # basta pra "GESTO: NNN" ou "NENHUM"
            },
        )

        raw = response["response"].strip().upper()
        logger.info(f"LLM bruto: '{raw}'")

        # O modelo explicitamente não encontrou gesto
        if "NENHUM" in raw:
            return None

        # Extraí o ID e valida contra a tabela real
        match = re.search(r"GESTO:\s*(\d+)", raw)
        if match:
            gid = int(match.group(1))
            if gid in GESTOS: # <- aceita a biblioteca inteira (0-20, 100-120)
                return gid
            logger.warning(f"LLM retornou ID {gid} fora da tabela GESTOS.")

    except Exception as e:
        logger.error(f"Erro ao chamar LLM: {e}")
        return None

def _classificar_por_keywords(self, texto: str) -> Optional[int]:
    """Fallback por palavras-chave (correspondência por palavra inteira)."""
    texto_lower = texto.lower()
    for gid, (_, keywords) in GESTOS.items():
        for kw in keywords:
            # Ignora keywords de 1 caractere: "i", "a", "o" casariam dentro
            # de qualquer palavra ("oi", "a mão") e geram falsos positivos.
            if len(kw) < 2:
                continue
            # \b garante palavra/expressão inteira, não substring solta.
            if re.search(rf"\b{re.escape(kw)}\b", texto_lower):
                logger.info(f"Keyword '{kw}' -> gesto {gid}")
                return gid
    return None

async def processar_comando(self, texto_usuario: str) -> Tuple[Optional[list], Optional[str],
float]:
    """
    Retorna (lista_de_ids, descricao, confianca).
    A lista sempre tem no máximo 1 gesto (1 sinal por frase).
    """
    if not texto_usuario or not texto_usuario.strip():
        return None, None, 0.0

    texto = texto_usuario.strip()

    # Estágio 1: LLM
    gid = await self._classificar_com_llm(texto)
    if gid is not None:
        desc = GESTOS[gid][0]
        logger.info(f"LLM -> gesto {gid}: '{desc}'")
        return [gid], desc, 1.0

    # Estágio 2: fallback por palavras-chave
    logger.warning("LLM não retornou gesto. Tentando fallback por keywords.")
    gid_fallback = self._classificar_por_keywords(texto)
    if gid_fallback is not None:
        desc = GESTOS[gid_fallback][0]
        return [gid_fallback], desc, 0.75

```

```
logger.warning(f"Nenhum gesto reconhecido para: '{texto}')"
return None, None, 0.0
```

Arquivo: backend/bluetooth_sender.py.

```
from bleak import BleakClient
import asyncio

MAC_ADDRESS = "b4:3a:45:b4:48:11"
UUID_COMANDO = "19b10001-e8f2-537e-4f6c-d104768a1214"
ble_client = None

# Mude para False quando for testar com o Arduino físico!
MOCK_BLE = False

async def conectar_bluetooth():
    """Função chamada quando o servidor liga para abrir a conexão."""
    global ble_client

    if MOCK_BLE:
        print("SIMULAÇÃO ATIVA: Fingindo conexão com o Arduino (Mock Mode).")
        return

    print(f"Tentando conectar ao MAO-BOT em {MAC_ADDRESS}...")
    ble_client = BleakClient(MAC_ADDRESS)
    try:
        await ble_client.connect()
        if ble_client.is_connected:
            print("Conectado ao MAO-BOT com sucesso! Mantendo conexão ativa.")
    except Exception as e:
        print(f"Aviso: Não foi possível conectar na inicialização. Erro: {e}")

async def desconectar_bluetooth():
    """Função chamada quando o servidor desliga para liberar o rádio do PC."""
    global ble_client
    if MOCK_BLE:
        return
    if ble_client and ble_client.is_connected:
        await ble_client.disconnect()
        print("Desconectado do MAO-BOT.")

async def enviar_comando_bluetooth(comando: int):
    """Função que o server.py chama para enviar o número do gesto."""
    global ble_client

    if MOCK_BLE:
        print(f"SIMULAÇÃO: Comando {comando} transmitido para a mão!")
        return True

    # Sistema de Autocura (Auto-reconnect)
    if not ble_client or not ble_client.is_connected:
        print("Bluetooth offline. Tentando reconectar agora...")
        try:
            if not ble_client:
                ble_client = BleakClient(MAC_ADDRESS)
            await ble_client.connect()
            print("Reconexão bem-sucedida!")
        except Exception as e:
            print(f"Falha na reconexão: {e}")
            return False # Retorna falso para o server.py saber que falhou

    # Transmissão do Comando
    try:
        await ble_client.write_gatt_char(UUID_COMANDO, bytearray([comando]))
        print(f"Comando {comando} transmitido!")
        return True
    except Exception as e:
        print(f"Falha na transmissão: {e}")
        return False
```

Arquivo: backend/requirements.txt.

```
annotated-doc==0.0.4
annotated-types==0.7.0
```

```
anyio==4.14.1
bleak==3.0.2
certifi==2026.6.17
click==8.4.2
dbus-fast==5.0.22
fastapi==0.138.1
h11==0.16.0
httpcore==1.0.9
httpx==0.28.1
idna==3.18
Levenshtein==0.27.3
ollama==0.6.2
pydantic==2.13.4
pydantic_core==2.46.4
python-Levenshtein==0.27.3
RapidFuzz==3.14.5
starlette==1.3.1
thefuzz==0.22.1
typing-inspection==0.4.2
typing_extensions==4.15.0
uvicorn==0.49.0
```

APÊNDICE C - FIRMWARE EMBARCADO (C++)

*Hardware de destino: Arduino Uno R4 WiFi. Firmware principal (BLE + servos).
Arquivo: arduino/mao_robotica/mao_robotica.ino.*

```
#include <ArduinoBLE.h>
#include <Servo.h>

// --- Configuração do Bluetooth (BLE) ---
BLEService maoService("19b10000-e8f2-537e-4f6c-d104768a1214");
BLEByteCharacteristic comandoCharacteristic("19b10001-e8f2-537e-4f6c-d104768a1214", BLERead |
BLEWrite);

// --- Configuração dos Servomotores ---
Servo mindinho;
Servo anelar;
Servo meio;
Servo indicador;
Servo polegar;
Servo pulso;

void setup()
{
    Serial.begin(9600);

    // Opcional: Aguarda o monitor serial abrir para debugar
    // while (!Serial);

    // Anexando os servos aos pinos digitais do Arduino
    pulso.attach(7);
    mindinho.attach(8);
    anelar.attach(9);
    meio.attach(10);
    indicador.attach(11);
    polegar.attach(12);

    // Começa com a mão aberta
    abrirMao();

    // --- Inicialização do BLE ---
    if (!BLE.begin())
    {
        Serial.println("Falha ao iniciar o Bluetooth!");
        while (1)
        {
            ;
        }
    }

    BLE.setLocalName("MaoRobotica");
    BLE.setAdvertisedService(maoService);

    maoService.addCharacteristic(comandoCharacteristic);
    BLE.addService(maoService);

    // Valor inicial da característica
    comandoCharacteristic.writeValue(5); // Começa no 5 (Mão aberta)
    Serial.print("Endereço MAC atual: ");
    Serial.println(BLE.address());
    BLE.advertise();
    Serial.println("Bluetooth ativo! Aguardando o backend conectar...");
}

void loop()
{
    // Fica escutando se o backend se conectou
    BLEDevice central = BLE.central();

    if (central)
    {
        Serial.print("Backend conectado! Endereço MAC: ");
        Serial.println(central.address());

        // Enquanto o backend estiver conectado
        while (central.connected())
        {
            ;
        }
    }
}
```

```

    // Verifica se o backend Python enviou um novo gesto
    if (comandoCharacteristic.written())
    {

        byte comandoRecebido = comandoCharacteristic.value();

        Serial.print("Comando recebido do backend: ");
        Serial.println(comandoRecebido);

        // Repassa o número recebido para a sua função de controle dos servos
        executarComando(comandoRecebido);
    }

    // Pequeno atraso para manter a estabilidade da conexão de rádio BLE
    delay(10);
}

Serial.print("Backend desconectado: ");
Serial.println(central.address());
Serial.println("Aguardando nova conexão...");
}
}

// =====
// FUNÇÕES DE CONTROLE DOS MOTORES (LIBRAS)
// =====

void executarComando(byte comando) {
    // O comando 120 cabe num 'byte' sem problemas (limite é 255)
    byte base = comando % 100;
    bool oscilar = comando >= 100;

    Serial.print("Executando base: ");
    Serial.print(base);
    Serial.print(" | Oscilar pulso: ");
    Serial.println(oscilar ? "SIM" : "NAO");

    // 1. Resolve a configuração física dos dedos
    if (base <= 17) {
        configurarDedos(base);
        delay(500); // Tempo para os motores estáticos chegarem na posição
    } else if (base == 18) {
        libras_agua();
    } else if (base == 19) {
        aspas();
    } else if (base == 20) {
        configurarDedos(5);
        oscilarPulso();
    }
}

// delay(1200);      // Aguarda um tempo de leitura confortável

// // 2. Resolve o movimento do pulso
// if (oscilar) {
//     oscilarPulso();
// } else {
//     pulso.write(100); // Fixa o pulso na posição de descanso
//     delay(1500);      // Aguarda um tempo de leitura confortável
// }

delay(800); // Aguarda um tempo de leitura confortável
// 3. Ao finalizar a ação, volta à posição de descanso
abrirMao();
}

void configurarDedos(byte base) {
    // ESTADO PADRÃO: MÃO TOTALMENTE FECHADA (Comando 0)
    // ---
    // Lógica Invertida (Hardware):
    // Mindinho, Anelar, Meio -> Fechado = 180 | Aberto = 0
    // Indicador -> Fechado = 0 | Aberto = 180
    // ---
    int min = 180;
    int ane = 180;
    int mei = 180;
    int ind = 0;
    int pol = 0;
}

```

```

// Muta os ângulos apenas para os dedos que precisam abrir/dobrar
switch (base) {
  case 1: ind = 180; break; // Indicador
  case 2: ind = 180; mei = 0; break; // Paz
  case 3: ind = 180; mei = 0; ane = 0; break; // Três
  case 4: ind = 180; mei = 0; ane = 0; min = 0; break; // Quatro
  case 5: ind = 180; mei = 0; ane = 0; min = 0; pol = 180; break; // Cinco / Mão Aberta
  case 6: ind = 180; min = 0; pol = 180; break; // Eu te amo
  case 7: pol = 180; break; // Joinha
  case 8: mei = 0; break; // Dedo do meio
  case 9: min = 0; pol = 180; break; // Shaka
  case 10: ind = 180; min = 0; break; // Rock
  case 11: min = 0; break; // Letra I
  case 12: ind = 180; pol = 180; break; // Letra L
  case 13: mei = 0; ane = 0; min = 0; break; // OK (Pol e Ind fechados, tocando)
  case 14: min = 80; ane = 100; mei = 140; ind = 70; pol = 120; break; // Letra C
  case 15: pol = 80; break; // Letra A (Punho fechado, polegar relaxado na lateral)
  case 16: min = 100; ane = 120; mei = 150; ind = 0; pol = 25; break; // Letra O
  case 17: mei = 0; ind = 180; pol = 80; break; // Base H
}

// Envia todos os comandos de uma vez
mindinho.write(min);
anelar.write(ane);
meio.write(mei);
indicador.write(ind);
polegar.write(pol);
}

void abrirMao() {
  configurarDedos(5); // CM-005 é a mão espalmada
  pulso.write(100);
}

void moverSuave(Servo &s, int alvo, int passo, int atraso) {
  int atual = s.read(); // ultimo angulo comandado
  // Exibe "Atual: " e depois o valor da variável
  //Serial.print("Atual: ");
  //Serial.println(atual);

  // Exibe "Alvo: " e depois o valor da variável
  //Serial.print("Alvo: ");
  //Serial.println(alvo);

  if (atual < alvo) {
    for (int a = atual; a <= alvo; a += passo) { s.write(a); delay(atraso); }
  } else {
    for (int a = atual; a >= alvo; a -= passo) { s.write(a); delay(atraso); }
  }
  s.write(alvo);
}

void oscilarPulso() {
  for (int i = 0; i < 2; i++) {
    moverSuave(pulso, 0, 2, 10); // sobe suave
    moverSuave(pulso, 160, 2, 10); // desce suave
  }
}

// =====
// MOVIMENTOS COM REPETIÇÃO NOS DEDOS
// =====

void libras_agua() {
  mindinho.write(180);
  anelar.write(180);
  meio.write(180);
  polegar.write(180);
  pulso.write(100);
  delay(300);

  // Indicador batendo repetidas vezes
  for (int i = 0; i < 4; i++) {
    indicador.write(0);
    delay(600);
    indicador.write(180);
    delay(600);
  }
}

```

```
    }  
}  
  
void aspas() {  
    mindinho.write(180);  
    anelar.write(180);  
    polegar.write(0);  
    pulso.write(100);  
  
    // Médio e Indicador dobram ao mesmo tempo  
    for(int i = 0; i < 2; i++) {  
        meio.write(0);          // Aberto  
        indicador.write(180); // Aberto  
        delay(700);  
        meio.write(180);       // Fechado  
        indicador.write(0);    // Fechado  
        delay(1000);  
    }  
  
    meio.write(0);          // Aberto  
    indicador.write(180);  
}
```

APÊNDICE D - FIRMWARE DE TESTES POR SERIAL (C++)

Hardware de destino: Arduino Uno R4 WiFi. Firmware auxiliar para testes via Monitor Serial. Arquivo: arduino/sinais_mao_robotica.ino.

```
#include <Servo.h>

Servo mindinho;
Servo anelar;
Servo meio;
Servo indicador;
Servo polegar;
Servo pulso;

void setup() {
  Serial.begin(9600);

  mindinho.attach(8);
  anelar.attach(9);
  meio.attach(10);
  indicador.attach(11);
  polegar.attach(12);
  pulso.attach(7);

  // Começa com a mão aberta
  abrirMao();

  Serial.println("Digite um numero de 0 a 10:");
  Serial.println("0 = Mao fechada");
  Serial.println("1 = Um");
  Serial.println("2 = Dois");
  Serial.println("3 = Tres");
  Serial.println("4 = Quatro");
  Serial.println("5 = Cinco / Mao aberta");
  Serial.println("6 = Eu te amo");
  Serial.println("7 = Joia");
  Serial.println("8 = Dedo meio");
  Serial.println("9 = Surfista");
  Serial.println("10 = Rock");
  Serial.println("11 = Aceno");
  Serial.println("12 = I em Libras");
  Serial.println("13 = L em Libras");
  Serial.println("14 = ok");
  Serial.println("15 = nao");
  Serial.println("16 = agua");
  Serial.println("17 = Aspas");
  Serial.println("18 = C em Libras");
  Serial.println("19 = H em Libras");
  Serial.println("20 = A em Libras");
  Serial.println("21 = 0 em Libras");
}

void loop() {
  if (Serial.available() > 0) {
    int comando = Serial.parseInt();

    // Limpa o restante do buffer do Serial
    while (Serial.available() > 0) {
      Serial.read();
    }

    executarComando(comando);
  }
}

void executarComando(int comando) {
  switch (comando) {
    case 0:
      fecharMao();
      Serial.println("Mao fechada");
      break;

    case 1:
      um();
  }
}
```



```
    Serial.println("Um");
    break;

case 2:
    dois();
    Serial.println("Dois");
    break;

case 3:
    tres();
    Serial.println("Tres");
    break;

case 4:
    quatro();
    Serial.println("Quatro");
    break;

case 5:
    cinco();
    Serial.println("Cinco ou Mao Aberta");
    break;

case 6:
    eu_te_amo();
    Serial.println("Eu Te Amo");
    break;

case 7:
    joia();
    Serial.println("Joia");
    break;

case 8:
    dedo_meio();
    Serial.println("Xingamento");
    break;

case 9:
    surfista();
    Serial.println("Surfista");
    break;

case 10:
    rock();
    Serial.println("Rock");
    break;

case 11:
    aceno();
    Serial.println("Aceno");
    break;

case 12:
    libras_I();
    Serial.println("I em Libras");
    break;

case 13:
    libras_L();
    Serial.println("L em Libras");
    break;

case 14:
    ok();
    Serial.println("ok");
    break;

case 15:
    nao();
    Serial.println("nao");
    break;

case 16:
    libras_agua();
    Serial.println("agua");
    break;
```

```

    case 17:
        aspas();
        Serial.println("Aspas");
        break;

    case 18:
        libras_C();
        Serial.println("C em Libras");
        break;

    case 19:
        libras_H();
        Serial.println("H em Libras");
        break;

    case 20:
        libras_A();
        Serial.println("A em Libras");
        break;

    case 21:
        libras_0();
        Serial.println("0 em Libras");
        break;

    default:
        erro();
        Serial.println("Comando invalido. Digite um numero de 0 a 21.");
        break;
}
}

// Função para abrir a mão
void abrirMao() {
    cinco();
}

// Função para fechar a mão
void fecharMao() {
    mindinho.write(180);
    anelar.write(180);
    meio.write(180);
    indicador.write(0);
    polegar.write(0);
    // pulso.write(100);

    delay(5000);
    abrirMao();
}

void um() {
    mindinho.write(180);
    anelar.write(180);
    meio.write(180);
    indicador.write(180);
    polegar.write(0);
    // pulso.write(100);

    delay(2000);
    abrirMao();
}

void dois() {
    mindinho.write(180);
    anelar.write(180);
    meio.write(0);
    indicador.write(180);
    polegar.write(0);
    // pulso.write(100);

    delay(2000);
    abrirMao();
}

void tres() {
    mindinho.write(180);

```

```

    anelar.write(0);
    meio.write(0);
    indicador.write(180);
    polegar.write(0);
    // pulso.write(100);

    delay(2000);
    abrirMao();
}

void quatro() {
    mindinho.write(0);
    anelar.write(0);
    meio.write(0);
    indicador.write(180);
    polegar.write(0);
    // pulso.write(100);

    delay(2000);
    abrirMao();
}

void cinco() {
    mindinho.write(0);
    anelar.write(0);
    meio.write(0);
    indicador.write(180);
    polegar.write(180);
    pulso.write(140);

    delay(2000);
}

void eu_te_amo() {
    mindinho.write(0);
    anelar.write(180);
    meio.write(180);
    indicador.write(180);
    polegar.write(180);
    // pulso.write(100);

    delay(2000);
    abrirMao();
}

void joia() {
    mindinho.write(180);
    anelar.write(180);
    meio.write(180);
    indicador.write(0);
    polegar.write(180);
    // pulso.write(100);

    delay(2000);
    abrirMao();
}

void dedo_meio() {
    mindinho.write(180);
    anelar.write(180);
    meio.write(0);
    indicador.write(0);
    polegar.write(0);
    // pulso.write(100);

    delay(2000);
    abrirMao();
}

void surfista() {
    mindinho.write(0);
    anelar.write(180);
    meio.write(180);
    indicador.write(0);
    polegar.write(180);
    delay(300);
    for (int i = 0; i < 3; i++) {

```

```

    pulso.write(30);
    delay(400);
    pulso.write(180);
    delay(400);
}

delay(2000);
abrirMao();
}

void rock() {
    mindinho.write(0);
    anelar.write(180);
    meio.write(180);
    indicador.write(180);
    polegar.write(0);
    // pulso.write(100);

    delay(2000);
    abrirMao();
}

void aceno() {
    // Mão aberta, pulso oscila 4 vezes
    abrirMao();
    delay(300);
    for (int i = 0; i < 4; i++) {
        pulso.write(30);
        delay(400);
        pulso.write(180);
        delay(400);
    }

    delay(2000);
    abrirMao();
}

void libras_I() {
    mindinho.write(0);
    anelar.write(180);
    meio.write(180);
    indicador.write(0);
    polegar.write(0);

    delay(2000);
    abrirMao();
}

void libras_L() {
    mindinho.write(180);
    anelar.write(180);
    meio.write(180);
    indicador.write(180);
    polegar.write(180);

    delay(2000);
    abrirMao();
}

void ok() {
    mindinho.write(0);
    anelar.write(0);
    meio.write(0);
    indicador.write(0);
    polegar.write(0);

    delay(2000);
    abrirMao();
}

void nao() {
    mindinho.write(180);
    anelar.write(180);
    meio.write(180);
    indicador.write(180);
    polegar.write(0);
    delay(300);
}

```

```

    for (int i = 0; i < 3; i++) {
        pulso.write(30);
        delay(400);
        pulso.write(180);
        delay(400);
    }

    delay(2000);
    abrirMao();
}

void libras_agua() {
    mindinho.write(180);
    anelar.write(180);
    meio.write(180);
    polegar.write(180);
    delay(300);
    // indicador bate repetidas vezes
    for (int i = 0; i < 4; i++) {
        indicador.write(0);
        delay(700);
        indicador.write(180);
        delay(700);
    }

    delay(2000);
    abrirMao();
}

void aspas() {
    mindinho.write(180);
    anelar.write(180);
    meio.write(0);
    indicador.write(180);
    polegar.write(0);
    pulso.write(120);
    delay(1200);
    meio.write(180);
    indicador.write(0);
    delay(1200);
    meio.write(0);
    indicador.write(180);
    delay(1200);
    meio.write(180);
    indicador.write(0);
    delay(1200);
    meio.write(0);
    indicador.write(180);
    delay(2000);
    abrirMao();
}

void libras_C() {
    mindinho.write(80);
    anelar.write(100);
    meio.write(140);
    indicador.write(70);
    polegar.write(120);
    delay(2000);
    abrirMao();
}

void libras_H() {
    mindinho.write(180);
    anelar.write(180);
    meio.write(0);
    indicador.write(180);
    polegar.write(80);
    delay(1200);
    pulso.write(50);
    delay(1600);
    Serial.println("Chegeui aqui no H");
    pulso.write(180);
    delay(2000);
    Serial.println("Chegeui aqui 2 no H");
    abrirMao();
}

```

```

void libras_A() {
    mindinho.write(180);
    anelar.write(180);
    meio.write(0);
    indicador.write(180);
    polegar.write(80);
    delay(2000);
    abrirMao();
}

void libras_0() {
    mindinho.write(100);
    anelar.write(120);
    meio.write(150);
    indicador.write(0);
    polegar.write(25);

    delay(2000);
    abrirMao();
}

void erro(){
    // Tremor da mao
    for (int i = 0; i < 8; i++) {

        pulso.write(80);
        delay(300);

        if(i == 0){
            indicador.write(0);
            delay(100);
        }else if(i == 1){
            indicador.write(180);
            meio.write(180);
            delay(100);
        }else if(i == 2){
            meio.write(0);
            anelar.write(180);
            delay(100);
        }else if(i == 3){
            anelar.write(0);
            mindinho.write(180);
            delay(100);
        }else if(i == 4){
            mindinho.write(0);
            polegar.write(0);
            delay(100);
        }else if(i == 5){
            polegar.write(180);
            delay(100);
        }else if(i == 6){
            polegar.write(50);
            delay(100);
        }else if(i == 7){
            mindinho.write(50);
            delay(100);
        }
        pulso.write(145);
        delay(300);
    }

    delay(2000);
    // Volta ao normal
    abrirMao();
}

```

APÊNDICE E - PROTÓTIPO INICIAL DO SERVIDOR (FLASK)

Hardware de destino: computador (PC/notebook). Primeira versão do servidor (Flask + Ollama), mantida como referência histórica. Arquivo: server/app.py.

```
"""
Servidor Flask do projeto FalaComaMão (LIBRAS-BOT).

Etapa 1 – orquestração entre Flutter e Ollama:

    Flutter (texto NLP) —HTTP—> /api/comando —> Ollama —> sinal escolhido
                                                |
                                                —> (etapa 4) BLE —> Arduino

Endpoints:
GET /api/status          - saúde do servidor + Ollama
GET /api/sinais          - lista todos os sinais cadastrados
POST /api/sinais         - cadastra um novo sinal
DELETE /api/sinais/<id>   - remove um sinal pelo id
POST /api/comando        - recebe texto e devolve qual sinal o Ollama escolheu
"""

from __future__ import annotations

import json
import logging
import re
import threading
from pathlib import Path
from typing import Any

from flask import Flask, abort, jsonify, request
from flask_cors import CORS

from ollama_client import OllamaError, classificar_intencao, healthcheck

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
)
log = logging.getLogger("falacomamao")

SIGNS_FILE = Path(__file__).parent / "signs.json"
_signs_lock = threading.Lock() # protege leitura/escrita concorrente do JSON

DEDOS = ("polegar", "indicador", "medio", "anelar", "minimo")
SERVOS = (*DEDOS, "pulso")
ID_REGEX = re.compile(r"^[a-z0-9_\-]{1,40}$")

app = Flask(__name__)
CORS(app) # permite chamadas do Flutter Web; em mobile é inofensivo.

# -----
# Persistência (JSON file)
# -----

def carregar_sinais() -> list[dict]:
    with _signs_lock:
        if not SIGNS_FILE.exists():
            return []
        with open(SIGNS_FILE, "r", encoding="utf-8") as f:
            data = json.load(f)
        return data.get("sinais", [])

def salvar_sinais(sinais: list[dict]) -> None:
    with _signs_lock:
        with open(SIGNS_FILE, "w", encoding="utf-8") as f:
            json.dump({"sinais": sinais}, f, ensure_ascii=False, indent=2)

# -----
# Validação
# -----
```

```

def _validar_sinal(payload: Any) -> dict:
    """
    Valida o payload de um novo sinal e devolve o dict normalizado.
    Levanta `ValueError` com mensagem amigável se algo estiver errado.
    """
    if not isinstance(payload, dict):
        raise ValueError("Esperado um objeto JSON com os campos do sinal.")

    sid = str(payload.get("id", "")).strip().lower()
    nome = str(payload.get("nome", "")).strip()
    descricao = str(payload.get("descricao", "")).strip()
    sinonimos = payload.get("sinonimos", [])
    angulos_raw = payload.get("angulos", {})

    if not sid or not ID_REGEX.match(sid):
        raise ValueError(
            "Campo 'id' inválido. Use minúsculas, números, hífen ou underscore (1-40 caracteres)."
        )
    if not nome:
        raise ValueError("Campo 'nome' é obrigatório.")
    if not descricao:
        raise ValueError("Campo 'descricao' é obrigatório (o Ollama usa pra decidir).")
    if not isinstance(sinonimos, list) or not all(isinstance(x, str) for x in sinonimos):
        raise ValueError("Campo 'sinonimos' deve ser uma lista de strings.")
    if not isinstance(angulos_raw, dict):
        raise ValueError("Campo 'angulos' deve ser um objeto com os 6 servos.")

    angulos: dict[str, int] = {}
    for servo in SERVOS:
        if servo not in angulos_raw:
            raise ValueError(f"Faltando ângulo do servo '{servo}'.")
        try:
            valor = int(angulos_raw[servo])
        except (TypeError, ValueError) as e:
            raise ValueError(f"Ângulo de '{servo}' precisa ser inteiro.") from e
        if not 0 <= valor <= 180:
            raise ValueError(f"Ângulo de '{servo}' fora do intervalo 0..180.")
        angulos[servo] = valor

    return {
        "id": sid,
        "nome": nome,
        "descricao": descricao,
        "sinonimos": [s.strip() for s in sinonimos if s.strip()],
        "angulos": angulos,
    }

# -----
# Endpoints
# -----

@app.route("/api/status", methods=["GET"])
def status():
    ollama = healthcheck()
    sinais = carregar_sinais()
    return jsonify({
        "servidor": "online",
        "sinais_cadastrados": len(sinais),
        "ollama": ollama,
    })

@app.route("/api/sinais", methods=["GET"])
def listar_sinais():
    return jsonify({"sinais": carregar_sinais()})

@app.route("/api/sinais", methods=["POST"])
def cadastrar_sinal():
    try:
        novo = _validar_sinal(request.get_json(silent=True))
    except ValueError as e:
        return jsonify({"erro": str(e)}), 400

    sinais = carregar_sinais()

```



```

    if any(s["id"] == novo["id"] for s in sinais):
        return jsonify({"erro": f"Já existe um sinal com id '{novo['id']}'"}, 409)

    sinais.append(novo)
    salvar_sinais(sinais)
    log.info("Sinal cadastrado: %s", novo["id"])
    return jsonify({"ok": True, "sinal": novo}), 201

@app.route("/api/sinais/<sinal_id>", methods=["DELETE"])
def remover_sinal(sinal_id: str):
    sid = sinal_id.strip().lower()
    sinais = carregar_sinais()
    novos = [s for s in sinais if s["id"] != sid]
    if len(novos) == len(sinais):
        abort(404, description=f"Sinal '{sid}' não encontrado.")
    salvar_sinais(novos)
    log.info("Sinal removido: %s", sid)
    return jsonify({"ok": True, "id": sid})

@app.route("/api/comando", methods=["POST"])
def comando():
    """
    Recebe o texto do Flutter (form-urlencoded `comando=...` OU JSON `{"comando": ...}`),
    pergunta pro Ollama qual sinal usar e devolve o sinal + ângulos.
    """
    texto = (
        request.form.get("comando")
        or request.get_json(silent=True) or {}).get("comando")
        or ""
    ).strip()

    if not texto:
        return jsonify({"erro": "Campo 'comando' vazio."}), 400

    sinais = carregar_sinais()
    if not sinais:
        return jsonify({
            "erro": "Nenhum sinal cadastrado. Cadastre sinais em /api/sinais antes."
        }), 409

    try:
        escolhido = classificar_intencao(texto, sinais)
    except OllamaError as e:
        log.exception("Falha no Ollama")
        return jsonify({"erro": str(e)}), 502

    if escolhido is None:
        return jsonify({
            "texto": texto,
            "sinal": None,
            "mensagem": "Nenhum sinal cadastrado corresponde ao que você falou.",
        })

    # TODO etapa 4: enviar `escolhido["angulos"]` para o Arduino via BLE.
    return jsonify({
        "texto": texto,
        "sinal": escolhido,
        "mensagem": f"Executando sinal: {escolhido['nome']}",
    })

@app.errorhandler(404)
def _404(e):
    return jsonify({"erro": str(e.description) if hasattr(e, "description") else "Não encontrado"}), 404

if __name__ == "__main__":
    # host=0.0.0.0 → o Flutter no celular acessa pelo IP do PC na mesma rede.
    app.run(host="0.0.0.0", port=5000, debug=True)

```

Arquivo: `server/ollama_client.py`.

```

"""
Cliente Ollama para o projeto FalaComaMão (LIBRAS-BOT).

Recebe uma fala/texto em português e a lista de sinais cadastrados,
e devolve qual sinal melhor representa a intenção do usuário.

A API do Ollama roda localmente em http://localhost:11434.
Usamos `format=json` + `temperature=0.1` para forçar saída estruturada
e determinística – ideal para classificação de intenção.
"""

from __future__ import annotations

import json
import logging
from typing import Optional

import requests

logger = logging.getLogger(__name__)

OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL = "llama3.1:8b"
TIMEOUT_SEGUNDOS = 60

class OllamaError(Exception):
    """Erro genérico ao falar com o Ollama (offline, modelo ausente, JSON inválido, etc.)."""

def _formatar_lista_sinais(sinais: list[dict]) -> str:
    """Monta a descrição dos sinais disponíveis para o LLM."""
    linhas = []
    for s in sinais:
        sinonimos = ", ".join(s.get("sinonimos", [])) or "(sem sinônimos)"
        linhas.append(
            f'- id: "{s["id"]}"\n'
            f' nome: {s["nome"]}\n'
            f' quando usar: {s["descricao"]}\n'
            f' sinônimos/contextos: {sinonimos}'
        )
    return "\n".join(linhas)

def _montar_prompt(texto_usuario: str, sinais: list[dict]) -> str:
    """Prompt de classificação de intenção em PT-BR."""
    lista = _formatar_lista_sinais(sinais)
    return (
        "Você é um classificador de intenções para uma mão robótica que executa sinais (gestos).
\n"
        "Sua tarefa: dada uma fala/texto do usuário em português, escolha QUAL DOS SINAIS abaixo "
        "melhor representa a intenção dele.\n\n"
        "SINAIS DISPONÍVEIS:\n"
        f"{lista}\n\n"
        "REGRAS:\n"
        '- Responda APENAS em JSON válido com o formato: {"sinal": "id_do_sinal"}.\n'
        '- Se NENHUM sinal se aplicar, responda: {"sinal": "nenhum"}.\n'
        '- O campo "sinal" deve conter EXATAMENTE um dos ids listados acima, ou "nenhum".\n'
        '- Não inclua explicações, comentários ou texto fora do JSON.\n\n'
        f'FALA DO USUÁRIO: "{texto_usuario}"\n\n'
        "JSON:"
    )

def classificar_intencao(
    texto_usuario: str,
    sinais: list[dict],
    *,
    model: str = MODEL,
    url: str = OLLAMA_URL,
    timeout: int = TIMEOUT_SEGUNDOS,
) -> Optional[dict]:
    """
    Classifica o texto do usuário e devolve o dicionário do sinal escolhido
    (com `id`, `nome`, `angulos`, ...), ou `None` se nenhum sinal corresponder.

    Levanta `OllamaError` se a chamada falhar ou se a resposta for inválida.
    """

```

```

"""
if not texto_usuario or not texto_usuario.strip():
    return None
if not sinais:
    logger.warning("Nenhum sinal cadastrado – Ollama não tem o que escolher.")
    return None

prompt = _montar_prompt(texto_usuario.strip(), sinais)

payload = {
    "model": model,
    "prompt": prompt,
    "stream": False,
    "format": "json",
    "options": {
        "temperature": 0.1,
        "top_p": 0.9,
    },
}

try:
    resp = requests.post(url, json=payload, timeout=timeout)
    resp.raise_for_status()
except requests.exceptions.ConnectionError as e:
    raise OllamaError(
        "Não consegui conectar no Ollama em "
        f"{url}. O serviço está rodando? (Tente `ollama list` no PowerShell.)"
    ) from e
except requests.exceptions.Timeout as e:
    raise OllamaError(
        f"Ollama demorou mais de {timeout}s para responder. "
        "Modelo pode estar carregando – tente de novo em alguns segundos."
    ) from e
except requests.exceptions.HTTPError as e:
    raise OllamaError(f"Ollama retornou HTTP {resp.status_code}: {resp.text}") from e

try:
    raw = resp.json().get("response", "")
    parsed = json.loads(raw)
except (ValueError, KeyError) as e:
    raise OllamaError(
        f"Resposta do Ollama não veio em JSON válido: {resp.text!r}"
    ) from e

sinal_id = (parsed.get("sinal") or "").strip().lower()
logger.info("Ollama escolheu: %r (texto=%r)", sinal_id, texto_usuario)

if not sinal_id or sinal_id == "nenhum":
    return None

for s in sinais:
    if s["id"].lower() == sinal_id:
        return s

# O modelo respondeu um id que não existe no signs.json – trata como nenhum.
logger.warning(
    "Ollama retornou id desconhecido %r (sinais disponíveis: %s)",
    sinal_id,
    [s["id"] for s in sinais],
)
return None

def healthcheck(*, url: str = OLLAMA_URL.replace("/api/generate", "/api/tags"),
                model: str = MODEL) -> dict:
    """
    Checa se o Ollama está no ar e se o modelo desejado está instalado.
    Retorna um dict com `online: bool` e `modelo_instalado: bool`.
    """
    info = {"online": False, "modelo_instalado": False, "modelos": []}
    try:
        resp = requests.get(url, timeout=5)
        resp.raise_for_status()
        info["online"] = True
        modelos = [m.get("name", "") for m in resp.json().get("models", [])]
        info["modelos"] = modelos
        info["modelo_instalado"] = any(m.startswith(model) for m in modelos)
    
```

```
except requests.exceptions.RequestException:  
    pass  
return info
```